

Verifiable Atomic Task Graph Relation Repair and Planning-Level Parallel Planning for Dual-Unit Collaborative Tasks

Abstract

Dual-unit collaborative planning requires checkable task dependencies and occupancy constraints. Candidate plans provided by large models, rules, or users may conform to a schema yet lack state support, resource ordering, or cooperative pairing, preventing their direct use in parallel scheduling. This study proposes a deterministic framework for atomic task graph relation repair. Typed nodes and relations represent discrete states, capacity-constrained resources, and execution-unit occupancy. The framework sequentially performs state closure, synchronization pairing, resource orientation, and conservative order compression. An accepted plan is returned only after both the joint graph audit and actual schedule checks pass; the edit log supports replay using accepted events. The experiments compare relation restoration, repair outcomes on identical inputs, and simulated execution of accepted plans using fixed skills. On controlled samples, full-label edge F1 increases from 0.9018 to 0.9813, but the full method does not outperform simple repair with shared compression. Public resource-task projections show benefits from resource orientation, whereas public tool tasks show increased formal acceptance alongside reduced reference-edge agreement. In dual-Panda physics simulation, skills execute successfully under nominal conditions but can fail under stress, showing that passing the discrete checks does not guarantee physical execution success. The project page is <https://roboplanner.github.io/atg-repair-planning/>.

Keywords: task planning , atomic task graph , causal closure , planning-level parallel scheduling , resource ordering , dual execution units

1 Introduction

Collaborative task planning requires converting candidate actions into checkable state dependencies, resource ordering, and execution-unit occupancy. Large language and vision-language models provide general language and multimodal representations [3,

49]. Embodied models map visual scenes and natural language to actions or high-level skills [13, 21, 69], and robotic planning methods can directly generate high-level task plans [35, 52]. Programmatic, affordance-based, and feedback interfaces connect language models to skills or executable programs [22, 24, 27]. These generation interfaces still require relation and occupancy checks before scheduling.

Schema validity does not guarantee causal closure or compatible occupancy. Existing surveys describe generation, feedback, and execution mechanisms in large-model agents [23]. Feedback-based methods [40], world models [19, 37], adaptive planning [45], and scene-task graph evaluation [1] improve candidates through plan revision, consequence prediction, or structured representation. Classical symbolic planning relies on predefined action and state models such as PDDL [31], while independent plan validators check candidate plans under those models [20]. The input considered here lies between candidate generation and a complete symbolic model: the node schema is valid, but state support may be missing, resource conflicts may lack orientation, and cooperative and occupancy fields may be inconsistent.

Graph-based reasoning explicitly organizes intermediate units and their relations [6, 32, 46, 55], grammar-constrained decoding reduces malformed structures [38], and closed-loop feedback and self-verification studies reveal both the role and limitations of external checks [7, 43]. This study addresses a specific post-generation interface problem: which relation repairs are supported by existing fields, which candidates should be rejected, and which parallel relations may enter strict scheduling. Relation edits therefore require traceable evidence, and outputs must pass both graph and schedule checks.

The proposed interface checks candidate actions or skills [13, 21, 69] and programmatic plans [22, 24, 27] before downstream execution. Bimanual reinforcement learning and multi-robot coordination [10, 15] address richer execution constraints. Here, L and R denote only two abstract base execution units, and B denotes simultaneous occupancy of both L and R. This planning-level mapping does not include trajectories, collisions, or dynamics; extension to an arbitrary number of execution units remains to be evaluated.

Candidate generation and deterministic post-processing are separate steps (Fig. 1). Post-processing uses existing node fields to repair state support, synchronization pairing, and capacity-1 resource ordering before tentatively deleting conservative order edges. Once all six joint graph checks pass, strict list scheduling assigns tasks, and the resulting assignment, occupancy, and numerical intervals are checked. Any failure returns diagnostics only. These acceptance conditions apply regardless of the candidate source.

The experiments test whether repair restores relations required by existing fields, whether the full method improves on simple rules under the same input, audit, compression, and scheduling conditions, and whether accepted plans complete tasks in dual-arm simulation with fixed skills. Controlled reference graphs, public source constraints, and physical state records are used for these tests. Reference agreement, formal acceptance, and execution success are measured separately.

The main contributions are as follows.

(1) An input interface with nine-field nodes and four relation types provides separate checks for state support, resource ordering, cooperative pairing, and conservative ordering. B denotes joint occupancy of the two base execution units and is used to check the actual occupancy of cooperative tasks.

(2) A four-stage repair procedure is followed by six joint graph checks and strict list scheduling. Repair uses existing fields, and every accepted or rejected edit is logged with its reason. Accepted events can be replayed to reconstruct the refined graph. Unresolved relational obligations or invalid schedules return diagnostics only. The guarantee covers represented discrete states, capacity-1 resources, and L/R occupancy.

(3) Repair methods are compared on identical inputs and schedulers on identical accepted graphs. Public-specification projections and dual-arm physics simulation test the scope of the method. The experiments report restored relations, differences between simple rules and the full method, and successes and failures in fixed-skill execution.

2 Related work

2.1 Large models and task planning

Research on candidate planning includes interactive agents [14, 58], surveys of planning mechanisms [23], and studies of planning ability and agents [50, 54, 57]. Search and tree-based reasoning explore candidate solutions [36, 59, 66, 68], reflection revises intermediate plans using feedback [40], and world models predict future states or evaluate plan consequences [19, 37]. These methods provide mechanisms for candidate generation and revision.

Language and multimodal models [3, 29, 34, 48, 49] support high-level robotic planning [35, 52]. Affordances and interactive feedback [22, 24], together with scene and skill interfaces [17, 22, 24], provide execution semantics. The present study checks fields and relations already represented in a candidate; it does not infer missing scene facts.

RT-2 [69], PaLM-E [13], and embodied multimodal models [21] investigate perception-to-skill generation; the present method verifies only explicit planning fields.

2.2 Structured representations and independent plan validation

Robot program generation [27], feedback-based adaptive planning [45], and scene-task graph evaluation [1] provide different forms of candidates. Graph-based reasoning [6, 30, 32, 46, 55, 60] organizes intermediate relations, while independent plan validators [20] check candidate semantics under a given model. The present method adds field-supported relation repair before validation and records the reasons for each modification.

Graph-based reasoning [6, 46, 55, 60] and asynchronous, tabular, and dynamic graph representations [8, 28, 44, 65] organize intermediate structures. Here, each relation type is required to have independently checkable semantics.

Structured decoding [38] constrains output form, but state support, resource ordering, and goals still require independent checks.

Verification and feedback outside the generator [7, 43] support separating acceptance decisions from candidate generation.

Graph learning and robust representations [18, 26, 47, 51, 63] focus on trainable representations; the present method instead bases its edits on explicit fields.

2.3 Dual-unit collaborative planning and structured plan generation

Action, affordance, and program generation [13, 24, 27, 69], together with vision-language representations [2], can provide candidates for collaborative tasks but do not directly specify repair obligations for typed relations.

Multitask manipulation, bimanual benchmarks, imitation learning, and diffusion policies [9, 11, 16, 25, 41] address action or control sequences.

Bimanual reinforcement learning [10], multi-robot coordination [15], dual-arm adaptive learning [12], and mobile bimanual control [64] address execution problems. In this study, L/R/B denotes planning-level occupancy only.

Programmatic plans [42], feedback-based reflection [22, 40], and graph-based reasoning [6, 46, 55] can all supply candidates to this interface, whose acceptance conditions are determined consistently by explicit fields and the actual schedule.

Industrial collaborative planning often optimizes assignment and scheduling over a given assembly structure. Nourmohammadi et al. jointly consider tasks, agents, layout, and operational constraints [33]; You et al. combine And-Or task graphs with optimized assignment [61]; and Zheng et al. address heterogeneous collaborative robots and limited resources [67]. The present method operates upstream of this optimization layer by repairing and auditing candidate relations. The MILP comparison on fixed accepted graphs evaluates scheduling gaps only; it does not reproduce the complete settings of those industrial systems.

Plan-over-Graph maps textual tasks to abstract graphs and parallel schedules [62], while graph-enhanced asynchronous reasoning analyzes plan dependencies [28]. The present method checks whether a plan can be accepted after relation repair. State support must carry exact atomic labels, and synchronization must pair with existing support. Edits require evidence from node fields, and compression tentatively deletes only conservative order edges. Plans are returned after all graph and actual occupancy checks pass, with both accepted and rejected events retained. In Section 4.8, the current heuristic does not improve on simple repair with shared compression across 160 controlled inputs. Section 4.9 tests the conditions under which it helps on public resource-constraint projections.

3 Methods

Figure 1 comprises representation, relation repair, and audit and output modules. Four stages transform the candidate G_0 into G_4 . Strict scheduling is permitted only after the joint graph audit passes; B occupies L and R over the same interval.

Section 3.1 defines the representation and assumptions, Section 3.2 specifies deterministic repair and scheduling, and Section 3.3 describes the executable interface, conditional properties, complexity, and failure behavior.

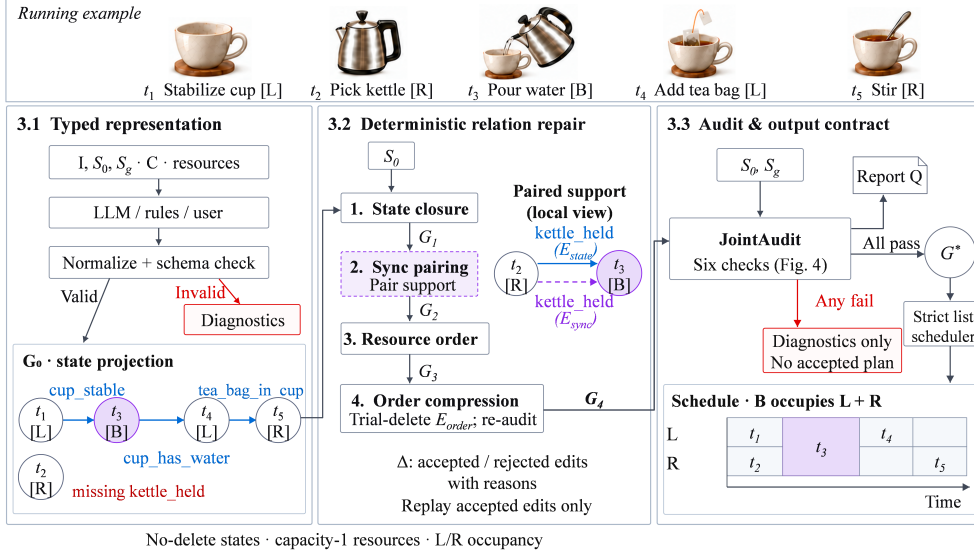


Fig. 1. Framework for verifiable atomic task graph relation repair and planning-level parallel planning. Explanatory schematic: initial visual drafts and object illustrations were produced with OpenAI ImageGen (backend model version not recorded); the diagram was subsequently reconstructed and edited in PowerPoint.

The model uses a delete-free discrete symbolic state abstraction, exclusive capacity-1 resource labels, and abstract execution-unit occupancy. All verification and scheduling claims below are conditional on these assumptions.

Candidate-generator interface. Given a task instruction, discrete initial and goal states, and a description of scene resources, the interface receives atomic nodes with the fields defined in Eq. (2). The formal acceptance entry point requires S_0 and S_g to be supplied explicitly and returns diagnostics if either is absent. Explicit empty sets are allowed, but omitted preconditions are not presumed to hold. The Python entry point receives sets of strings; a serialization adapter must validate state lists before converting them to sets. Convenience inference in lower-level checkers is used only for diagnostics and is excluded from formal acceptance. Candidates may originate from a large language model, a rule system, or a user. Missing fields, incorrect raw types, or illegal values yield parsing diagnostics. Type checking precedes normalization: identifiers, actions, objects, and collection elements must be strings; collection fields must be lists; and durations must be finite positive numbers rather than Boolean values. The experiments instantiate this interface with deterministic controlled candidates and directly generated GPT-6 plans. Model conditions and input derivation rules are given in Section 4.5 and Appendix A of the supplementary material.

For a task instance $X=(I, S_0, S_g, C, E_{base}, X_{occ})$, I denotes the instruction; S_0 and S_g are finite sets of discrete state atoms; C is the scene context; $E_{base}=\{L, R\}$ comprises the two base execution units; and $X_{occ}=\{L, R, B\}$ is the occupancy-label domain. Let $R_{src}=\cup_i Res_i$ be the finite set of resource labels occurring in the instance. If the system

supplies a separate resource whitelist, the schema audit also requires R_{src} to be a subset of that whitelist. The method returns a typed ATG G , an audit report Q , and a planning-level schedule π . B denotes simultaneous occupancy of L and R and does not introduce a third base execution unit.

Each node v_i records an identifier, action, object, nonempty precondition and postcondition sets, a positive duration estimate d_i , a finite resource-label set Res_i , a nonempty set of candidate occupancy labels $Cand_i$, and a mode $mode_i$. An empty Res_i means that the node declares no exclusive capacity-1 resource; it does not establish the absence of resource occupancy in an open world. A state-support relation carries a causal state label. A synchronization relation is a joint-start readiness marker paired with existing state support for a cooperative node and cannot replace that support. A resource-mutex relation carries a resource label. Only conservative order edges are eligible for compression.

Two nodes may execute concurrently only if they are incomparable in the directed precedence projection, have disjoint represented resources and assigned execution-unit occupancy, and satisfy all incoming synchronization requirements. These conditions are computed solely from the supplied symbolic fields and estimated durations.

3.1 Atomic task graph representation for verifiable parallel planning

Task atomization maps a high-level instruction to typed symbolic operations. In the beverage example, stabilizing the cup, picking up the kettle, pouring water, adding a tea bag, and stirring explicitly expose state transitions, estimated durations, resource labels, and candidate execution-unit occupancy. Parallel eligibility is then computed from these fields rather than inferred from wording alone.

The ATG extends a precedence graph with typed state-support and synchronization relations, together with discrete resource and execution-unit fields. It serves as the intermediate representation for the implemented verifier; it does not cover all semantics available in HTN, STN, PDDL, or resource-constrained scheduling models.

Figure 2 shows the nine-field node, the labeled state projection, and the four relation types. $Cand$ specifies candidate occupancy; synchronization is paired with existing state support by exactly matching source, target, and state.

3.1.1 Atomic task nodes

A candidate plan is normalized into an atomic task graph with typed edges:

$$G_{ATG} = (V, E_{state}, E_{mutex}, E_{sync}, E_{order}) \quad (1)$$

G_{ATG} is a directed multirelation with typed edges. Both E_{state} and E_{sync} contain (source,target,state) triples, but serve different purposes. E_{state} establishes producer-consumer state support. E_{sync} is paired with E_{state} having the same source, target, and state only when the target node is cooperative, marking joint-start readiness. E_{mutex} contains (source,target,resource) triples, whereas E_{order} contains directed node pairs. Path and acyclicity checks use the directed node-pair projection after state

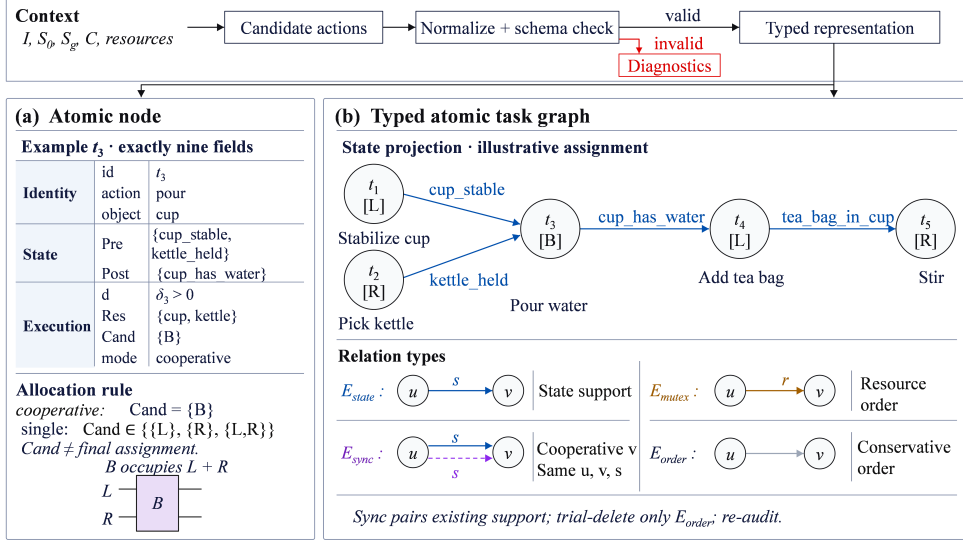


Fig. 2. Atomic task graph construction and node attribute definitions. Explanatory schematic: the initial visual draft was produced with OpenAI ImageGen (backend model version not recorded); the diagram was subsequently reconstructed and edited in PowerPoint.

and resource labels are discarded. A paired E_{sync} does not introduce an additional state-production fact.

Each node is defined as

$$v_i = \langle id_i, action_i, object_i, Pre_i, Post_i, d_i, Res_i, Cand_i, mode_i \rangle \quad (2)$$

where id_i is unique; $action_i$ and $object_i$ are nonempty finite strings denoting the action and object; Pre_i and $Post_i$ are nonempty finite sets; Res_i is a finite, possibly empty subset of R_{src} ; and $d_i > 0$ is the supplied duration estimate. The mode satisfies $mode_i \in \{\text{single}, \text{cooperative}\}$. If $mode_i = \text{cooperative}$, then $Cand_i = \{B\}$; if $mode_i = \text{single}$, then $Cand_i$ is a nonempty subset of $\{L, R\}$. The implementation validates only $E_{base} = \{L, R\}$; support for an arbitrary number of base execution units is not claimed.

An explicit state-support edge is defined by state production and consumption:

$$EligibleSupport(v_j, v_i, s) \Leftrightarrow j \neq i \wedge s \in Post_j \cap Pre_i \quad (3)$$

E_{state} contains selected support relations rather than all eligible producer-consumer pairs. For each noninitial precondition, the selected edge must originate from an eligible producer that can precede the consumer without introducing a cycle.

$$Closed(v_i, k) \Leftrightarrow Pre_i \subseteq Available(k-1) \wedge \forall s \in Pre_i \setminus S_0, \exists u \in Executed(k-1) : (u, v_i, s) \in E_{state} \quad (4)$$

Let $Executed(k-1)$ denote the set of nodes that have passed closure checks and executed before the k th topological position, with $Available(0) = S_0$. In topological

order, a node v_i must pass its closure check before $Post_i$ is added to Available. This update is monotonic: the current model excludes delete effects, consumable facts, fact invalidation, and mutually exclusive state atoms. Problems requiring these semantics fall outside the verifier's guarantee.

Goal reachability requires S_g to be a subset of the final Available set after all accepted nodes execute. This criterion is valid only under the same delete-free state assumption.

The representation therefore distinguishes a state label occurring in the graph from a state supported by an executable predecessor. It cannot establish the semantic correctness of omitted or incorrectly labeled facts.

3.1.2 Resource occupancy and candidate execution-unit constraints

Parallel eligibility also depends on resource and execution-unit occupancy. Every declared resource label denotes an exclusive, nonshareable, capacity-1 discrete resource throughout the node's scheduled interval. An empty resource set indicates the absence of input labels, not the absence of physical resource requirements. Read/write access modes, capacities greater than 1, continuous geometry, and collision constraints are not represented.

The resource mapping of a node is

$$\rho(v_i) = Res_i, \quad Res_i \subseteq R_{src} \quad (5)$$

Res_i may contain object, tool, or discrete-workspace labels. The positive value d_i is supplied with the node. The scheduler uses it to construct the half-open interval $[start_i, finish_i)$, but does not estimate physical duration.

The resource compatibility condition for two nodes is

$$Compat_R(v_i, v_j) \Leftrightarrow Res_i \cap Res_j = \emptyset \quad (6)$$

$Cand_i$ is supplied through the candidate interface rather than derived from geometry. Schema violations can be detected, but formally well-formed yet incorrect resource or execution-unit labels may remain undetected without external evidence. Such cases are measured only against the controlled reference graphs.

$$Occ(L) = \{L\}, \quad Occ(R) = \{R\}, \quad Occ(B) = \{L, R\} \quad (7)$$

A node assigned to B or marked cooperative occupies L and R over the same interval. It cannot overlap a node assigned to L, R, or B.

Let $Support(v_i)$ collect the existing valid state-support relations directed into v_i , namely all E_{state} triples (u, v_i, s) with $s \in Post_u \cap Pre_i$. Let $SyncIn(v_i)$ collect incoming synchronization relations for v_i , represented as E_{sync} triples. For a cooperative node, $SyncComplete(v_i)$ holds if and only if $SyncIn(v_i) = Support(v_i)$. It is always true for a single node, although the schema audit rejects any E_{sync} directed into such a node. Initial states need no newly created support; however, any retained state support into a cooperative node must also be paired by the same source, target, and state. E_{sync} marks cooperative readiness and cannot replace causal support.

$$\begin{aligned}
ParallelEligible(v_i, v_j) \Leftrightarrow & Incomparable_G(v_i, v_j) \wedge Compat_R(v_i, v_j) \\
& \wedge [\exists x_i \in Cand_i, x_j \in Cand_j : Occ(x_i) \cap Occ(x_j) = \emptyset] \\
& \wedge SyncComplete(v_i) \wedge SyncComplete(v_j)
\end{aligned} \tag{8}$$

Equation (8) defines static parallel eligibility at graph level, not actual temporal overlap. *IncomparableG* means that neither node has a directed path to the other. Resource compatibility, execution-unit occupancy, and synchronization completeness exclude three corresponding classes of static conflicts. Actual start times and execution-unit availability are determined by strict list scheduling in Eqs. (15) and (16).

Equations (6)–(8) define conditional planning-level eligibility under the supplied discrete fields. They do not cover continuous collisions, geometry, or unannotated resource semantics.

3.2 Deterministic relation repair and strict list scheduling

A schema-valid candidate graph G_0 undergoes state closure, synchronization pairing, resource orientation, and conservative order compression in that order. Figure 3 shows the four local transformations. The original candidate G_0 , refined graph, and structured event ledger are used in six deterministic checks; the refined graph is accepted as G^* only if these checks pass. Execution-unit assignment and strict list scheduling are described in Section 3.2.2, and the formal acceptance interface in Section 3.3.1. Missing required fields or illegal values stop the procedure before repair. Even when fields are formally valid, reference data may still be needed to determine whether their semantics are correct.

Figure 3 shows four local relation transformations and scheduling after the audit. The u, v resource and order examples are local illustrations. Compression trial-deletes only valid E_order edges and re-audits each deletion.

3.2.1 Dependency repair and static parallel-eligibility auditing

Closure is checked in a deterministic topological order. Ties are resolved by a canonical key comprising the numeric interpretation and full identifier, independently of the input order of nodes or edges. A noninitial precondition is accepted only when an executed producer connects to its consumer through an E_{state} relation carrying that exact state label. E_{sync} cannot replace this support.

$$\begin{aligned}
& Available(0) = S_0 \\
Closed(v_i, k) \Rightarrow & Available(k) = Available(k-1) \cup Post_i \\
\neg Closed(v_i, k) \Rightarrow & Available(k) = Available(k-1)
\end{aligned} \tag{9}$$

A cycle causes immediate closure-audit failure. A node with an unavailable state, or with only an implicit producer and no typed support edge, does not execute; its postconditions are not added to *Available*.

Nodes that fail the check contribute no postconditions. Subsequent nodes can use only initial states and valid production facts from previously executed nodes.

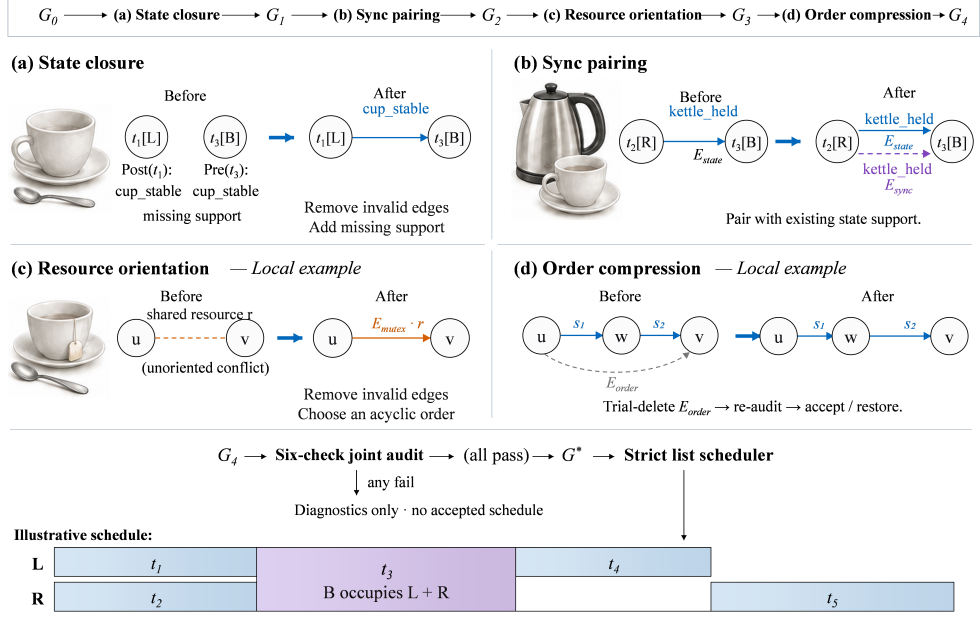


Fig. 3. Deterministic ATG edits with complete relation labels and joint auditing. Explanatory schematic: initial visual drafts and object illustrations were produced with OpenAI ImageGen (backend model version not recorded); the diagram was subsequently reconstructed and edited in PowerPoint.

State closure first removes mislabeled E_{state} relations whose labels are absent from the intersection of the source Post and target Pre, recording each deletion and its justification. For every missing support obligation (v_i, s) , it then constructs the eligible-producer set $P_i(s)$, excluding candidates whose insertion would introduce a cycle. Let $G_{p,i,s}$ denote the current graph after inserting $(p, v_i, E_{state, state=s})$.

$$p^* = \underset{\text{subject to } Acyclic(G_{p,i,s})}{\operatorname{argmin}}_{p \in P_i(s)} (L_{cp}(G_{p,i,s}), key(p)) \quad (10)$$

State obligations are processed in canonical $(key(v_i), s)$ order, and Eq. (10) resolves producer ties deterministically using $key(p)$. This greedy rule preserves local acyclicity for every accepted insertion, but is incomplete: it may preclude an alternative producer combination that would yield a feasible closure. Thus, the absence of an eligible candidate or failure of the final audit means only that this algorithm rejects the current refinement; it does not prove that no other feasible ATG exists.

After synchronization pairing, the resource stage removes mislabeled E_{mutex} relations whose resource labels are absent from the intersection of the endpoint Res sets, records the reasons, and then checks capacity-1 resource conflicts between incomparable node pairs.

$$F = \{(v_i, v_j) \mid Incomparable_G(v_i, v_j) \wedge Res_i \cap Res_j \neq \emptyset\} \quad (11)$$

Node pairs are processed in canonical $(key(v_i), key(v_j))$ order. Comparability is recomputed on the updated graph at every step, and shared resource labels are recorded in lexicographic order. Both orientations are tested for each remaining conflict. Orientations that introduce cycles are discarded; the others are compared lexicographically by critical-path length, source-node key, and target-node key. Let G_{dir} denote the graph after adding the resource relations in the selected direction.

$$dir^* = \underset{subject\ to\ Acyclic(G_{dir})}{argmin_{dir \in \{v_i \rightarrow v_j, v_j \rightarrow v_i\}}} (L_{cp}(G_{dir}), key(source(dir)), key(target(dir))) \quad (12)$$

For every cooperative node, the procedure rebuilds SyncIn to equal Support exactly. Each existing E_{state} support is paired with an E_{sync} having the same source, target, and state. Orphaned or mislabeled E_{sync} relations and those targeting single nodes are removed. Synchronization completeness is checked by set equality. During scheduling, a cooperative node is assigned to B and occupies L and R simultaneously. This mechanism represents joint-start readiness for two execution units rather than a general barrier language independent of causal edges.

The state and resource stages remove only mislabeled relations unsupported by node fields and add relations justified by existing evidence. This semantic cleanup differs from conservative order compression of valid E_{order} . The procedure does not reconstruct omitted precondition, resource, or candidate execution-unit labels.

3.2.2 Conservative order compression and strict list scheduling

For any acyclic graph G , let $Paths(G)$ be the paths in its directed relation projection. The critical-path length is defined as

$$L_{cp}(G) = \max_{p \in Paths(G)} \sum_{v_i \in p} d_i \quad (13)$$

Compression iterates over E_{order} using the complete canonical key (source, target, type, state, resource), independently of storage order. Each candidate edge is temporarily removed, after which the state/goal audit and unordered shared-resource audit are repeated. E_{state} , E_{sync} , and E_{mutex} are never compression candidates.

$$\begin{aligned} Removable(e) \Leftrightarrow & AuditStateGoal(G \setminus \{e\}).valid \\ & \wedge ResourceConflicts(G \setminus \{e\}) = \emptyset, \quad e \in E_{order} \end{aligned} \quad (14)$$

An edge is permanently removed only if Eq. (14) holds. E_{state} , E_{mutex} , and E_{sync} are never compression targets.

This procedure can release precedence supplied solely as conservative order. Repeated auditing prevents an accepted deletion from reintroducing represented state or resource violations.

The scheduler uses deterministic list scheduling rather than global makespan optimization. Nodes follow the same topological tie-breaking order. Let $E^{\rightarrow}$ denote the

directed node-pair projection of all relations. For node v_i , the release time r_i is the maximum completion time of all directed predecessors.

$$r_i = \max(\{finish_j \mid (v_j, v_i) \in E^{\rightarrow}\} \cup \{0\}) \quad (15)$$

$$\begin{aligned} a_i(x) &= \max(\{r_i\} \cup \{available(e) \mid e \in Occ(x)\}) \\ x_i^* &= \operatorname{argmin}_{x \in Cand_i} (a_i(x), key(x)) \\ start_i &= a_i(x_i^*), \quad finish_i = start_i + d_i \end{aligned} \quad (16)$$

For a cooperative node, $x_i^* = B$. After a node finishes, every base execution unit e in $Occ(x_i^*)$ is updated to $available(e) = finish_i$. Thus B updates both L and R , whereas a single node updates only its selected L or R unit. The fixed order $key(L) < key(R) < key(B)$ resolves ties only when earliest start times are equal.

The timeline is deterministic for a fixed graph and duration vector, but global makespan optimality is not guaranteed. The implementation uses double-precision floating-point arithmetic. An accepted output must have finite, nonnegative start and finish times, $finish_i > start_i$, and relative error between interval length and d_i no greater than 10^{-9} , with an absolute tolerance of 0. Precedence is checked directly as $finish_u \leq start_v$ without absolute-time slack; all aggregate values must be finite. These conditions jointly define the supported numerical scale, which does not include arbitrary positive real values. A duration of 10^{-17} is representable in isolation, but appending it after time 1 produces a zero-length interval through rounding and must yield only a numerical diagnostic.

Each accepted schedule record contains the node identifier, start and finish times, selected execution-unit occupancy, and resource labels. The accompanying report records schema, closure, goal, cycle, resource, synchronization, and compression decisions.

The final accepted graph satisfies checkable conditions under the represented fields. Executable nodes have explicit E_{state} support; every existing valid state support for a cooperative node has an exactly matching E_{sync} readiness marker, and the node occupies both base execution units. Shared declared capacity-1 resources are ordered, and the precedence projection is acyclic. These are enforced acceptance conditions rather than unconditional claims about unrepresented semantics.

3.3 Reusable algorithm modules, formal properties, and failure boundaries

The audit and scheduling interface takes the refined graph G_4 , original candidate G_0 , structured event ledger Δ , discrete states, and node attributes. Figure 4 shows how successful and failed audits are handled. The properties below follow from the representation assumptions, six acceptance tests, and scheduling rules.

Figure 4 separates replay of accepted events, the six-check joint graph audit, and the output paths. Failure of a graph or schedule check returns diagnostics only. Rejected events are archived but do not participate in graph reconstruction.

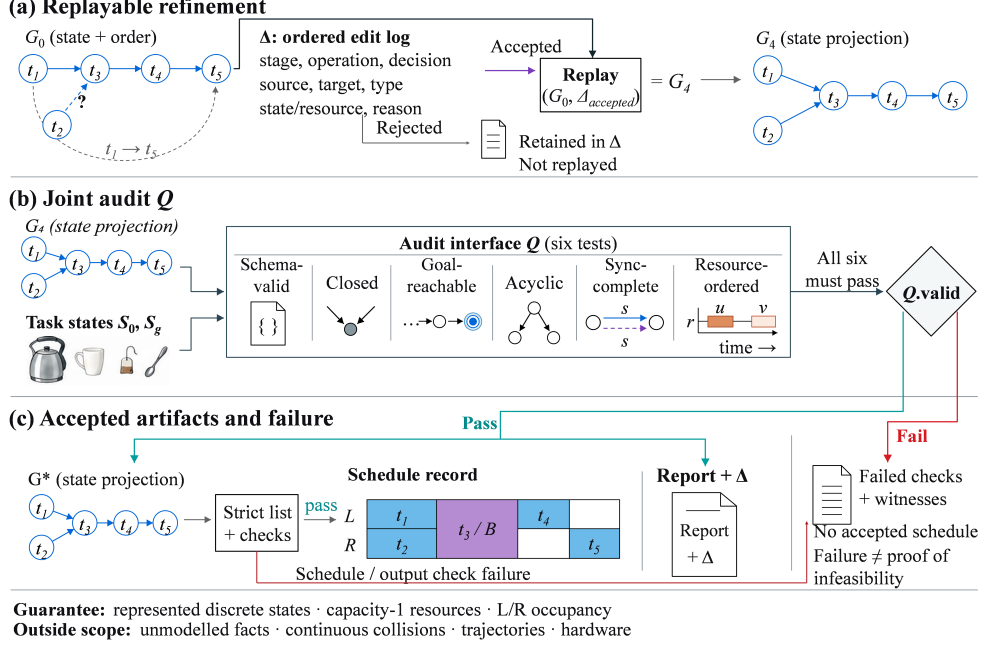


Fig. 4. Algorithmic audit interface, outputs, and failure boundaries. Explanatory schematic: initial visual drafts and object illustrations were produced with OpenAI ImageGen (back-end model version not recorded); the diagram was subsequently reconstructed and edited in PowerPoint.

3.3.1 Algorithm and audit interface

Algorithm 1 starts from a candidate ATG G_0 , initial states S_0 , and goal states S_g . After repair produces G_4 and Δ , the joint audit determines whether the graph is accepted as G^* . Success returns G^* , the graph audit, and a strict planning-level schedule. Failure returns diagnostics without an accepted schedule.

Algorithm 1 Deterministic ATG repair, auditing, and planning-level scheduling

Input: Explicitly supplied candidate ATG G_0 , initial states S_0 , and goal states S_g .

Output: Accepted graph G^* , audit report Q , and schedule π ; otherwise, a failure report.

- 1: Confirm that S_0 and S_g are explicit and correctly typed; validate required raw fields and their actual types, nonempty Pre/Post/Cand, consistency of mode and Cand, finite positive durations (reject Booleans), edge endpoints, relation types, and labels. Return parsing diagnostics on failure.
- 2: Normalize identifiers and initialize the event log. If the original directed projection is cyclic, return diagnostics without entering the four repair stages.
- 3: $G_1 \leftarrow \text{StateClosureRepair}(G_0, S_0)$. Remove mislabeled state relations, then select acyclic producers in canonical obligation order.
- 4: $G_2 \leftarrow \text{PairSynchronizationMarkers}(G_1)$. Rebuild each cooperative node's E_{sync} as the exactly label-matched pairing set of all existing valid E_{state} supports.
- 5: $G_3 \leftarrow \text{OrientResourceMutex}(G_2)$. Remove mislabeled resource relations, then determine acyclic orientations in canonical node-pair order.
- 6: $G_4 \leftarrow \text{CompressOrderEdges}(G_3)$. Re-audit every tentative deletion.
- 7: $Q \leftarrow \text{JointAudit}(G_4, S_0, S_g)$: schema, closure, goals, cycles, synchronization, and resources.
- 8: If $Q.\text{valid}=\text{false}$, then
- 9: Return failure report Q without an accepted strict schedule.
- 10: End if
- 11: $\pi \leftarrow \text{ListSchedule}(G_4, \text{Cand}, \text{mode}, \text{duration}, \text{strict}=\text{true})$. Check actual assignment, occupancy, strictly positive intervals, relative duration error, and finite aggregates. On failure, return diagnostics without an accepted schedule.
- 12: Preserve machine time precision when exporting the original candidate graph G_0 , refined graph, assignments and half-open intervals, resources, and accepted/rejected events with applicable labels. Round only for display.
- 13: Return G_4 , Q , π .

The formal interface fixes two base execution units with B as joint occupancy and rejects labels inapplicable to a relation type. After the joint graph audit passes, the actual assignment must still satisfy Cand, mode, and occupancy constraints.

In execution order, the audit log records the stage, edit operation (edge insertion or deletion), accept/reject decision, source, target, type, applicable state or resource label, and reason, together with diagnostics such as unresolved states. Replaying the accepted edits in order from G_0 reconstructs G_4 . Rejected events remain in the log, but their operations are not applied to the reconstructed graph. The log is not a complete search trace of every unselected orientation. Operations that delete and reconstruct the same synchronization edge are counted separately from net relation changes.

3.3.2 Conditional propositions, proof sketches, and complexity

Proposition 1 (Conditional closure). Under the delete-free state model, if the final joint audit returns $\text{valid}=\text{true}$, every accepted node has all preconditions at its topological position, and every noninitial precondition s has an incoming E_{state} from an executed producer with $\text{edge.state}=s$. Proof sketch: Eq. (4) checks available states and exact-label support node by node. Only a node that passes may add Post to the monotonic Available set. The result follows by induction over the topological order.

Proposition 2 (Represented discrete-resource safety). Under exclusive capacity-1 resources, an empty final ResourceConflicts set means that every otherwise incomparable node pair sharing a represented resource has been ordered. Proof sketch: Eq. (12) accepts only acyclic orientations, and list scheduling sets release times no earlier than the completion of all directed predecessors. The pair therefore cannot overlap. The proposition excludes omissions hidden by empty resource sets, continuous collisions, and capacities greater than 1.

Proposition 3 (Audit-preserving compression). Only E_{order} is trial-deleted, and state, goal, and resource audits are repeated after every deletion. Each accepted deletion therefore preserves these tested conditions. Proof sketch: induction over the deletion sequence; the current graph is updated only when the audit still passes. This proposition establishes neither a globally minimal remaining edge set nor makespan optimality.

Proposition 4 (Joint structural conditions of accepted outputs). Every inserted E_{state} or E_{mutex} is screened for cycles. Each cooperative node's E_{sync} set equals its state-support set exactly, and no single node has E_{sync} . The final directed projection is checked again for acyclicity, ordered resources, and synchronization. A graph with valid=true therefore satisfies these represented planning-level structural conditions; a failed graph does not enter strict scheduling. Together with half-open intervals, Occ occupancy updates, and actual-assignment checks, accepted schedules have no overlap on declared capacity-1 resources or on L/R execution units.

Complexity includes the total field size. Let $n \geq 1$ be the number of nodes, m the current number of relations, $K = 1 + |S_0| + |S_g| + \sum_i (|Pre_i| + |Post_i| + |Res_i| + |Cand_i|)$, b the maximum of 1 and the number of these field entries in any one node, and $q = \sum_i |Pre_i| + |S_g|$. Atomic labels and identifiers have bounded length, and hash-table operations have amortized constant cost. Reading and writing unbounded text or extended metadata are counted separately. Let A denote a conservative cost for one joint audit and its preprocessing.

$$A = K \log(1 + K) + (n + m)^2 + qmb + n^2[n + m + b \log(1 + b)] \quad (17a)$$

$$T = O((1 + nq + n^2 + h) A(n, M, K, b, q)) \quad (17b)$$

The first term of A covers input scanning and set sorting; the second covers relation preprocessing and topological operations; the third covers scans of incoming edges and Post lists for state obligations; and the last covers pairwise reachability and resource checks. Let m_0 be the initial edge count, $h = |E_order(G_0)|$, and M the maximum edge count during repair. Then $M \leq 2m_0 + 2q + bn(n-1)/2$. The state stage tests at most nq producers, the resource stage at most $n(n-1)$ orientations, and compression at most h tentative deletions. Graph copying, synchronization rebuilding, the final joint audit, strict scheduling, and output checks are covered by the same conservative unit cost, yielding Eq. (17b) for the complete procedure. This is a conservative asymptotic upper bound for the full procedure, not a tight bound or a statement about global solution optimality.

3.3.3 Outputs, experimental mapping, and failure boundaries

The interface exports the original candidate G_0 , refined graph G_4 , complete edit log, audit report, and schedule records that pass the checks. Failed instances retain their inputs, logs, and diagnostics without returning an accepted schedule.

Reference consistency and acceptance are interpreted separately. Node F1, full-label edge F1, and field accuracy measure agreement with controlled reference graphs, using the edge key (source,target,type,state,resource). Parallel precision P, recall R, and F1 compare predicted and reference node-pair sets from Eq. (8) and are sensitive to omitted or incorrect field labels. In contrast, closure rate, joint acceptance rate, and scheduled conflict rate are coupled to the implemented checks.

Passing therefore means only that the supplied fields satisfy the implemented acceptance tests. It does not establish that all task-relevant facts, resources, or execution-unit constraints have been supplied.

Accordingly, the experiments report reference consistency, acceptance outcomes, efficiency, and input-field perturbations as distinct categories of evidence.

4 Experiments and analysis

The experiments measure reference agreement, formal acceptance, and physical completion separately. Reference agreement compares the output with a given reference. Formal acceptance counts inputs that pass the represented constraint checks, whereas physical completion counts successful fixed-skill executions. Each metric has its own denominator and failure records for analyzing repair, comparisons on identical inputs, and simulated execution.

Sections 4.2–4.4 examine repair obligations using controlled references, ablations, and field perturbations. Sections 4.5–4.7 examine the conversational candidate interface, input sensitivity, and scheduling quality on identical graphs. Sections 4.8–4.9 use shared-compression controls and public specifications to assess when relation repair is useful. Section 4.10 connects the plans to physics simulation with fixed skills. Detailed regression matrices, derived-input results, and reproduction protocols are provided in the separate supplementary material.

4.1 Experimental setup and evaluation scope

The controlled dataset contains 160 records: 40 duration-perturbed variants from each of four fixed topology templates for beverage preparation, boxing, assembly, and sorting. Thus, there are 4 independent reference topologies, not 160. Durations use generation-seed=13 and jitter=0.2. Candidate-structure perturbations are separately determined by structure-seed=0, the record index, and published method-specific off-sets. Both sources of randomness and their scope are recorded in the experiment configuration file.

The same deterministic framework generates both the controlled reference graphs and candidate perturbations. This permits component-level tests but may favor the framework's ontology. These references are not independent human annotations or solutions from an external solver.

The text-order, direct-candidate, and schema-constrained-candidate conditions in the main experiment use the same task records but different preset perturbation configurations. They therefore cannot be interpreted as an algorithm ranking on identical candidates. State-relation deletion probabilities are 0.35, 0.28, and 0.14, respectively, and synchronization-relation deletion probabilities add 0.15 to the corresponding values. Deterministic changes to precondition, resource, execution-unit, and order fields are specified in Appendix A.5. The proposed method and the schema-constrained candidate form a paired comparison on identical inputs. The reference-graph serial control reads the controlled reference nodes and relations and adds a total order, giving it additional reference information. It provides a serial-time and validity baseline, rather than a relation-repair method with identical information access.

All conditions share the task records and use their own fixed candidate-construction configurations. Continuous structural and scheduling metrics are reported as per-record means and sample standard deviations. Parallel P, R, and F1 are pooled over node pairs in Table 2 and Fig. 5, but summarized per record in Table 4. Acceptance rates are the proportions of passing records. The 40 duration variants of a topology are not treated as independent topology replicates or used for significance inference. The scaling experiment has only 1 reference topology at each node count and constructs candidates using 6 structural perturbation seeds. Each instance–method combination is warmed up 10 times and timed 50 times; its median is then summarized by the mean and sample standard deviation across the 6 candidate instances.

The supplementary GPT-6 trials and the 160 controlled records are analyzed separately. The same-graph MILP uses exactly the same nodes, typed precedence relations, durations, resources, Cand, and mode as list scheduling and does not read an independent reference graph. It optimizes only assignment and timing on the accepted graph. Section 4.8 also evaluates publicly archived candidates from two models. That evaluation is not a cross-model ranking under unified generation conditions and has no independent human annotation of industrial procedures.

Formal pass rates in Sections 4.2–4.9 concern represented states, capacity-1 resources, and L/R occupancy. The procedural semantic correctness of model-generated content is interpreted separately from those formal checks. Section 4.10 additionally reports task completion, object positions, and contact observations in physics simulation.

OpenAI Codex assisted the preparation and revision of implementation, analysis, and plotting scripts. Reported quantitative results and data plots were obtained by executing the archived scripts on retained inputs and result files; Fig. 9 consists of frames from the recorded MuJoCo runs. Tool use for candidate generation is described separately in Section 4.5 and Supplementary Appendix A. Historical tool versions are stated only where retained in the project records.

4.2 Quality of typed relations and node fields

Node F1 compares sets of (id, action, object) identities. Full-label edge F1 compares the five-part keys (source, target, type, state, resource) of non-order relations, with nonapplicable fields left empty. State-field and execution-unit-field accuracy require

exact set equality for matching node identifiers. Because the current controlled candidates always retain every node, node F1 is 1.0000 in every condition and is not repeated in Table 1. It cannot serve as evidence of task-atomization performance.

Table 1. Atomic task graph structural quality of controlled candidate graphs.

Method	Full-label edge F1	State-field accuracy	Execution-unit-field accuracy
Text-order heuristic	0.7483 $\pm$ 0.1348	0.6500 $\pm$ 0.0290	0.8667 $\pm$ 0.0784
Direct candidate graph (controlled)	0.7948 $\pm$ 0.1278	0.6500 $\pm$ 0.0290	0.8250 $\pm$ 0.0145
Schema-constrained candidate graph (controlled)	0.9018 $\pm$ 0.0807	1.0000 $\pm$ 0.0000	0.8667 $\pm$ 0.0784
Verified ATG	0.9813 $\pm$ 0.0297	1.0000 $\pm$ 0.0000	0.8667 $\pm$ 0.0784

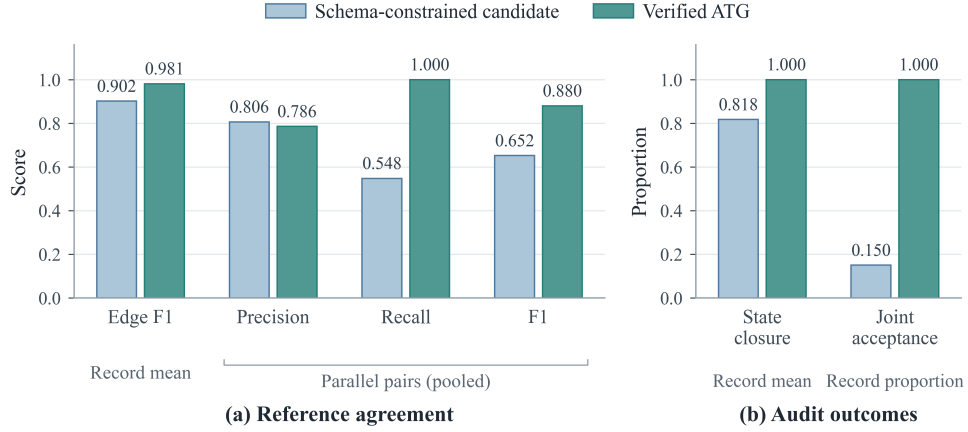


Fig. 5. Comparison between schema-constrained candidates and Verified ATGs. (a) Reference agreement. (b) Audit results. The 160 records comprise 40 duration variants of each of 4 fixed topologies. Edge F1 and closure rate are arithmetic means over all records; parallel P, R, and F1 are pooled over node pairs, and acceptance is a proportion of records. Bar heights show the stage-level aggregate for the same metric.

After repair of schema-constrained candidates, full-label edge F1 increases from 0.9018 to 0.9813. Node-pair-pooled parallel F1 increases from 0.6522 to 0.8800, and recall rises from 0.5477 to 1.0000, while precision falls from 0.8060 to 0.7857. Thus, covering more reference parallel pairs is still accompanied by additional node pairs that disagree with the reference.

Closure rate counts state obligations rather than nodes. Let $a_{i,s}$ indicate that, when node i is checked under Eq. (9), precondition s is available and, for a noninitial

state, has exact-label support from an executed node. Let A_final denote the final set of available states. Then

$$C_{closure} = \frac{\sum_i \sum_{s \in Pre_i} a_{i,s} + |S_g \cap A_{final}|}{\sum_i |Pre_i| + |S_g|} \quad (18)$$

The denominator is the total number of precondition obligations and goals; the rate is defined as 1 when the denominator is 0. If a node cannot execute, its satisfied precondition obligations still contribute to the numerator, but its Post is not added to the available states. For cyclic input, the execution check stops immediately, and the numerator retains only goals satisfied in S_0 . This continuous metric is reported separately from the Boolean Closed check among the six gates.

Mean causal closure rate increases from 0.8175 to 1.0000, and the joint acceptance proportion rises from 0.1500 (24/160) to 1.0000 (160/160). Both are coupled to the acceptance rules and cannot be interpreted as open-world accuracy. Edge F1 remains 0.9813, showing that passing the audit does not imply reproducing every reference edge. A synchronization edge must pair with an E_state edge having the same source, target, and state; it cannot replace state support.

4.3 Constraint-aware relation repair and parallel eligibility

Validation first checks whether outputs satisfy represented constraints and reports reference-relative parallel quality separately. The missing-dependency rate uses full-label non-order edges. Parallel-node-pair precision, recall, and F1 compare the predicted set defined in Eq. (8) with the controlled reference set. Joint acceptance, closure, and schedule conflicts are computed by the validator and scheduler.

Table 2. Missing dependencies, parallel reference agreement, and joint acceptance.

Method	Missing-dependency rate	Parallel P	Parallel R	Parallel F1	Pred./ref. pairs	Jointly accepted (/160)
Text-order heuristic	0.3845±0.1668	—	0.0000	0.0000	0/440	0
Direct candidate graph (controlled)	0.3230±0.1673	0.5859	1.0000	0.7389	751/440	0
Schema-constrained candidate graph (controlled)	0.1698±0.1239	0.8060	0.5477	0.6522	299/440	24
Reference-graph serial control	0.0000±0.0000	—	0.0000	0.0000	0/440	160
Verified ATG	0.0318±0.0495	0.7857	1.0000	0.8800	560/440	160

Table 2 presents both reference agreement and acceptance. After repair, the missing-dependency rate is 0.0318, all 160 inputs are accepted, and the schedule-conflict rate for declared resources is 0, but reference-relative parallel precision remains 0.7857. Order compression may expose additional static parallel pairs left by missing or mislabeled fields; passing the joint audit does not imply restoration of reference semantics. Single-fault regressions are reported in supplementary Table C1.

The fixed procedure first restores state support and synchronization obligations justified by fields, then orients resource conflicts, and finally determines whether conservative ordering can be removed from the repaired graph. Component-disablement experiments examine the role of each obligation; they do not provide evidence from comparisons of alternative stage orders.

The scheduling comparison reports efficiency and joint acceptance together. Timelines that fail the joint audit are used for diagnosis and are excluded from efficiency claims about accepted plans.

Planning efficiency is reported using critical-path length, estimated makespan, waiting time, execution-unit utilization, and the ratio of serial makespan to the current makespan. All values use symbolic durations and abstract occupancy.

Table 3. Planning-level list-scheduling results.

Method	Estimated makespan (symbolic units)	Planning-level speedup	Jointly accepted (/160)
Text-order heuristic	13.9736±2.5078	1.0000±0.0000	0
Direct candidate graph (diagnostic)	11.3809±2.3130	1.2383±0.1095	0
Schema-constrained candi- date graph (diagnostic)	12.7029±2.4336	1.1066±0.0858	24
Reference-graph serial con- trol	13.9736±2.5078	1.0000±0.0000	160
Verified ATG	11.3416±2.6257	1.2535±0.1347	160

Table 3 reports descriptive means and between-record sample standard deviations for the 40 duration variants of each of four fixed topologies; these variants are not independent topology replicates. The text-order and reference-graph serial controls impose the same total order and therefore have identical timeline-efficiency values in Table 3, although their node fields, relation completeness, and joint acceptance differ. Critical-path length, waiting time, utilization, closure, and conflict metrics are collected in supplementary Table A3.

In Fig. 6, the Verified ATG has a mean estimated makespan of 11.3416 and a mean per-record speedup of 1.2535 relative to the reference-graph serial control, whose mean makespan is 13.9736. All 160/160 plans are strictly accepted. Shorter timelines for other, unaccepted candidates are diagnostic only. This comparison examines the

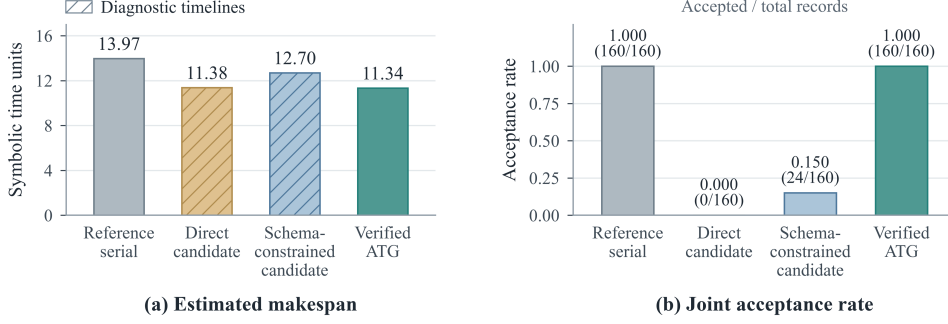


Fig. 6. Planning-level efficiency and joint acceptance. (a) Mean estimated duration for 160 records (symbolic time units); hatched bars denote diagnostic candidate timelines. (b) Joint acceptance and passing count/160. Reference serial denotes the reference-graph serial control. The Verified ATG has a mean speedup of 1.2535, obtained by averaging each record's serial-to-current duration ratio.

release of parallelism under the symbolic protocol; the incremental benefit of repair algorithms on identical inputs is examined separately in Section 4.8.

The ablations separately disable state-support repair, synchronization repair, resource-order repair, order compression, or execution-unit assignment, while retaining existing typed relations in the input candidate graph. Each condition reports joint graph acceptance and strict plan acceptance separately.

The ablations report full-label edge F1, missing-dependency rate, parallel-node-pair F1, joint/plan acceptance, and speedup. Nonstrict timelines that fail the joint audit are diagnostic only. Accepted-plan speedup is calculated only over the subset passing strict plan checks.

As shown in Table 4, disabling state-support, synchronization, and resource-order repair reduces joint acceptance to 0.4437, 0.4750, and 0.7375, respectively, corresponding to 71, 76, and 118 accepted plans. Without resource ordering, edge F1 is 0.9961, higher than the full method's 0.9813, but joint acceptance falls to 0.7375. Reproducing reference edges is therefore not equivalent to constraint validity. The conflict rate in this condition is 0.0437, compared with 0 in all other conditions. Disabling order-edge compression retains 160/160 acceptance but lowers both diagnostic and accepted-plan speedup to 1.1629. When execution-unit assignment is disabled, joint graph acceptance remains 1.0000, but strict plan acceptance is 0, showing that graph auditing cannot replace the assignment required for scheduling. The full method accepts 160/160 plans, with an accepted-plan speedup of 1.2535.

4.4 Reproducibility, input sensitivity, and failure boundaries

Regression tests check rejection of explicit violations, while field perturbations test detection of omitted semantics. Joint acceptance is 0 for all nine single-fault classes (supplementary Table C1), showing that the corresponding checks detect the tested

Table 4. Ablation of validation and scheduling components.

Component setting	Full-label edge F1	Missing dependency rate	Parallel F1 per record	Graph Plan		
				passes / 160	passes / 160	Accepted-plan speedup
Without state repair	0.8931±0.1130	0.1556±0.1498	0.6185±0.3280	71	71	1.2455±0.1323
Without synchronization repair	0.9393±0.0577	0.1065±0.0936	0.8667±0.1419	76	76	1.2747±0.1235
Without resource orientation	0.9961±0.0139	0.0074±0.0264	0.8812±0.2327	118	118	1.2091±0.0790
Without order-edge compression	0.9813±0.0297	0.0318±0.0495	0.5250±0.3279	160	160	1.1629±0.1745
Without unit assignment	0.9813±0.0297	0.0318±0.0495	0.8667±0.1419	160	0	—
Full method	0.9813±0.0297	0.0318±0.0495	0.8667±0.1419	160	160	1.2535±0.1347

Note: Continuous metrics are the mean $\pm$ sample standard deviation over 160 records. Parallel F1 is a per-record mean, unlike the node-pair-pooled F1 in Table 2 and Fig. 5. The graph-pass and plan-pass columns give the numbers passing the six graph gates and strict schedule checks, respectively, both with denominator 160. Accepted-plan speedup is computed only over passing subsets, whose sizes are 71, 76, 118, 160, 0, and 160 in row order. Means over different subsets are not directly ranked. Speedups for nonstrict diagnostic timelines are given in supplementary Table A4.

violations. Facts absent from the fields remain outside these checks. Historical timings for 4, 6, 8, and 10 nodes and their single-topology limitation are reported in supplementary Section C.1 and Fig. A1.

Field sensitivity is evaluated on the same 160 records using five deterministic perturbations: deleting an E_state relation, omitting a resource annotation, deleting a precondition state, ensuring a change to an occupancy label, and combining these perturbations. Each individual and combined perturbation actually changes 160/160 records. The execution-unit subperturbation also changes 160/160 records; itemwise comparisons with the original fields confirm that the labels differ.

Table 5 retains the main field-sensitivity results. Complete per-field dispersion and diagnostics with different counting units are provided in supplementary Tables A2 and B1. Missing relations may be reconstructed from remaining evidence, whereas omissions in precondition, resource, or occupancy fields themselves cannot be recovered by internal auditing.

The field perturbations below are controlled inputs. Section 4.5 separately reports directly generated GPT-6 candidates; derived variants are not counted as new model outputs.

Table 5(a) compares exact field-reference agreement and acceptance counts before and after repair. Table 5(b) compares the repaired relations and static parallel sets. Together, they examine the difference between formal acceptance and reference semantics.

Table 5(a). Field-reference agreement and joint acceptance under controlled field perturbations.

Candidate setting	State-field accuracy	Resource-field accuracy	Occupancy-field accuracy	Joint acceptance: before $\rightarrow$ after (each /160)
Unperturbed reference graph	1.0000	1.0000	1.0000	160 $\rightarrow$ 160
State-support edge deletion	1.0000	1.0000	1.0000	0 $\rightarrow$ 160
Missing resource annotation	1.0000	0.8250	1.0000	92 $\rightarrow$ 160
Precondition-state deletion	0.8250	1.0000	1.0000	12 $\rightarrow$ 160
Execution-unit label change	1.0000	1.0000	0.8250	160 $\rightarrow$ 160
Combined perturbation	0.8250	0.8250	0.8250	5 $\rightarrow$ 160

Note: Each setting contains 160 records, comprising 40 duration variants of each of 4 fixed topologies. Field accuracy is the proportion of nodes with matching identifiers whose field sets exactly equal the reference, averaged over records. Joint acceptance is the passing count/160. The unperturbed reference graph has a static-parallel-set Jaccard index of 1.0000.

The static-parallel-set Jaccard index compares eligible parallel node pairs in the separately repaired candidate and reference graphs; it does not measure agreement between complete plans or scheduled intervals.

After state-support edges are deleted, the remaining Pre/Post evidence suffices to restore the relations, yielding both full-label edge F1 and static-parallel-set Jaccard of 1.0000. Missing resource annotations, deleted preconditions, and changed occupancy labels instead involve missing or incorrect fields. Internal auditing cannot recover unobserved semantics from these fields. Field-accuracy dispersion is reported in supplementary Table A2.

When execution-unit labels are changed, joint acceptance remains 1.0000 both before and after repair, but static-parallel-set Jaccard is only 0.7443. Under combined perturbations, post-repair joint acceptance and closure rate are both 1.0000, whereas full-label edge F1, parallel-node-pair F1, and static-parallel-set Jaccard are 0.8419,

Table 5(b). Reference agreement of relations and static parallel sets after controlled field perturbations.

Controlled perturbation	Full-label edge F1	Parallel-node-pair F1	Static-parallel-set Jaccard
State-support edge deletion	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000
Missing resource annotation	0.9770 $\pm$ 0.0271	0.9500 $\pm$ 0.0869	0.9167 $\pm$ 0.1448
Precondition-state deletion	0.8698 $\pm$ 0.0686	0.9721 $\pm$ 0.1077	0.9609 $\pm$ 0.1472
Execution-unit label change	1.0000 $\pm$ 0.0000	0.8050 $\pm$ 0.2844	0.7443 $\pm$ 0.3065
Combined perturbation	0.8419 $\pm$ 0.0628	0.7193 $\pm$ 0.2802	0.6259 $\pm$ 0.3040

Note: Values are means $\pm$ sample standard deviations over 160 records and are descriptive summaries only, not statistical inference over independent topologies. All five perturbation classes actually change 160/160 records. The execution-unit sub-perturbation applies only to the final two classes and also changes 160/160 records in each. Results are from deterministic surrogate candidates; no actual API benchmark, model ranking, or significance test was conducted.

0.7193, and 0.6259, respectively. Thus, incorrect fields that satisfy the schema can still pass the joint gates; validation covers represented obligations only.

Independent variations in duration and relation retention are examined in Section 4.6. Supplementary Table C2 summarizes module inputs, acceptance conditions, evidence, and guarantee scope.

4.5 GPT-6 task planning and paired-input experiments

GPT-6 Astra directly authored 24 candidate plans across six task domains in a single conversation. The plans contain 254 nodes, with 7–16 nodes per task and 24 cooperative nodes distributed across 17 tasks. Task specifications were saved before the candidates, and all candidates were frozen before joint evaluation. One output was retained per task, without outcome-based revision or filtering. Because the conversation already contained the method semantics and validation code, this trial evaluates the generation–validation interface under method-informed conditions. Task specifications and candidates were both constructed by the same model and do not constitute an independent human reference or a blind test.

Both the original candidates and the fully post-processed outputs pass all six joint checks and strict scheduling in 24/24 cases. Exported timelines and log replays are consistent. The original relations comprise 266 state-support edges, 41 synchronization edges, and 54 resource-order edges, with no conservative order edges. Post-processing produces no net relation changes. The 82 accepted events correspond to deletion and reconstruction of 41 synchronization edges and are not counted as 82 repaired defects. Table 6 reports results by task domain. Relative to fully serial execution of the same

nodes, mean per-task speedup is 1.2411 and mean symbolic makespan reduction is 18.49%.

Table 6. Strict acceptance and planning-level efficiency of directly generated GPT-6 plans.

Task domain	Tasks	Nodes	Strict acceptance: before $\rightarrow$ after	Estimated makespan (symbolic units)	Speedup
Assembly	4	38	4/4 $\rightarrow$ 4/4	17.25 $\pm$ 3.50	1.2390 $\pm$ 0.1565
Packaging	4	43	4/4 $\rightarrow$ 4/4	14.75 $\pm$ 2.50	1.3034 $\pm$ 0.0662
Inspection	4	41	4/4 $\rightarrow$ 4/4	18.00 $\pm$ 4.32	1.1063 $\pm$ 0.0444
Warehousing	4	44	4/4 $\rightarrow$ 4/4	16.75 $\pm$ 5.32	1.2240 $\pm$ 0.2185
Sorting	4	43	4/4 $\rightarrow$ 4/4	13.25 $\pm$ 3.86	1.3546 $\pm$ 0.0872
Workstation preparation	4	45	4/4 $\rightarrow$ 4/4	23.50 $\pm$ 5.32	1.2193 $\pm$ 0.1301
Overall	24	254	24/24 $\rightarrow$ 24/24	17.25 $\pm$ 5.00	1.2411 $\pm$ 0.1402

Note: In the Overall row, task, node, and acceptance counts are totals, while makespan and speedup are means $\pm$ sample standard deviations over the 24 tasks. Acceptance is the strictly passing count/task count before $\rightarrow$ after repair. Estimated makespan uses symbolic units. Continuous metrics describe between-task means $\pm$ sample standard deviations, not uncertainty from repeated model sampling. All 24 original inputs and post-processed outputs pass, with 0 net relation changes; efficiency gains come from parallel scheduling on the same graph.

Four paired inputs are constructed from the 24 frozen node sets: all relations, nodes only, state support only, and linear conservative ordering. Components are then disabled individually (supplementary Table C3). The full method accepts 24/24 in all four groups. For nodes-only inputs, disabling synchronization or resource orientation reduces acceptance to 7/24 or 2/24, respectively. These derived inputs expose relational obligations; they are not samples of naturally generated errors.

For the 24 jointly accepted tasks with linear-order inputs, the full method deletes 175 conservative order edges and achieves a mean speedup of 1.2388 ± 0.1438 , compared with 1.0000 ± 0.0000 without compression. Nodes-only and all-relations inputs contain no conservative order edges, so disabling compression does not change their results. Fig. 8(a) shows per-task paired differences; the simple-repair control with shared compression is examined in Section 4.8.

4.6 Sensitivity to missing relations, durations, and branching structure

Relation-deletion and duration tests are derived from the 24 frozen candidates; the complete protocol and results are given in supplementary Table C4. Three deletion levels and three fixed seeds produce 216 relation variants. All are rejected before

repair and strictly accepted afterward, with all exported outputs passing independent checks. The 168 records with fixed relations and varied positive durations also pass. These results test relation reconstruction from remaining fields and acceptance stability within the tested duration range; they do not measure natural error rates or reference-edge recovery accuracy.

To vary branching structure rather than merely append a serial tail, branch widths of 2, 4, and 8 are combined with 2 and 4 processing layers, shared or separate tools, and single or cooperative joins, producing 24 programmatically constructed graphs. The node count equals the number of layers multiplied by (the branch count + one join node), giving actual sizes of 6, 10, 12, 18, 20, and 36 nodes. Initial graphs provide state and synchronization support but leave shared-tool conflicts unoriented. This group is a structural stress test, not new GPT-6 outputs or independent industrial tasks.

All 12 initial graphs with shared tools are rejected for unordered resources and pass after repair. All 12 graphs with separate tools pass both before and after repair. Their mean speedups are 1.0000 and 1.4870, respectively, showing that additional branches cannot bypass a capacity-1 bottleneck. Median post-processing times for the four 36-node configurations range from 108.06 to 554.27 ms (Fig. 7). Timing begins with an already loaded graph and includes repair, the joint audit, and strict scheduling; each graph has 3 warm-up runs and 10 measurements. Per-configuration values and the interpretation of single/B joins are given in supplementary Table C5.

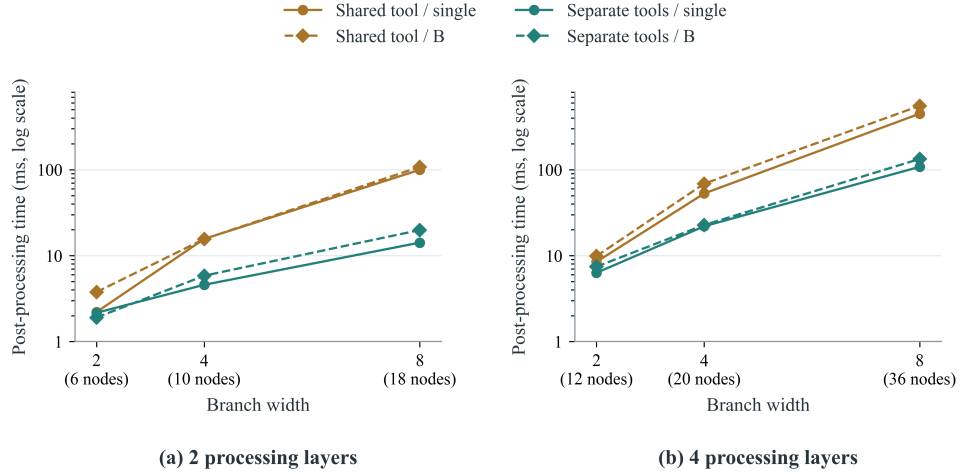


Fig. 7. Effect of branching structure and resource configuration on post-processing time. (a) 2 processing layers. (b) 4 processing layers. Each curve denotes one combination of shared/separate tools and single/B joins. Each point is the median of 10 measurements after 3 warm-up runs on the same graph; the vertical axis is logarithmic. Lines connect measured configurations only and do not represent asymptotic complexity.

4.7 Same-graph MILP scheduling comparison and case analysis

To evaluate scheduling quality separately, the graphs accepted by the full method for the 24 original GPT-6 plans are passed to both strict list scheduling and MILP. Both use identical nodes, all typed precedence relations, durations, capacity-1 resources, Cand, and mode. MILP optimizes only allowable execution-unit choices and temporal ordering; it neither reconstructs relations nor reads a reference graph. The milp interface in SciPy 1.15.3 invokes HiGHS [53], with a 10 s time limit per instance and a target relative gap of 0. Solver status, lower bounds, and raw solutions are saved.

The MILP minimizes makespan. Each edge in the accepted graph requires the predecessor to finish no later than the successor starts. Task pairs sharing a resource or selected base execution unit satisfy disjunctive nonoverlap constraints, and B occupies both units. Big M is the fully serial duration of the same nodes. Earliest start times are recomputed by forward propagation using the solver's integer assignments and pairwise orders to remove tiny overlaps caused by floating-point tolerances, followed by independent output checks. This step changes neither node fields nor the orders selected by the solver.

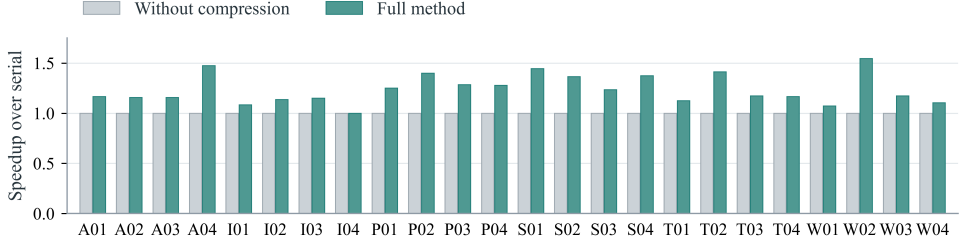
Table 7. List scheduling and MILP on the same 24 accepted graphs.

Scheduling method	Estimated makespan (symbolic units)	Attains same-graph optimum	Relative optimality gap, mean $\pm$ SD	Maximum gap
Strict list scheduling	17.2500 $\pm$ 4.9978	10/24	6.95% $\pm$ 8.59%	35.71%
MILP	16.1250 $\pm$ 4.5618	24/24	0.00%	0.00%

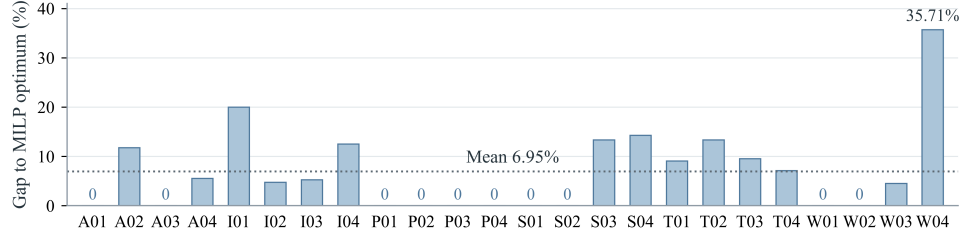
Note: Both schedules pass independent export checks in 24/24 cases. Makespan uses symbolic units, and continuous metrics are per-task means $\pm$ sample standard deviations. The relative gap is list-scheduling makespan divided by the MILP optimum on the same graph, minus 1. All 24 MILP runs return optimal status; after forward propagation, the feasible upper bound agrees with the solver lower bound within a relative tolerance of 1×10^{-6} . Optimality is restricted to the fixed accepted graph and does not assess globally optimal relation repair.

Mean makespan is 17.2500 for strict list scheduling and 16.1250 for MILP. The mean per-task optimality gap is 6.95%, the median is 5.01%, and 10/24 tasks attain the optimum on the same graph. The largest gap occurs for W04, three-source order kitting: list scheduling gives 19 and MILP gives 14, a gap of 35.71%. Strict acceptance does not imply minimum makespan; an accepted graph can still be passed to an optimizing scheduler. Fig. 8(b) reports the per-task gaps.

A01, bracket assembly, illustrates the complete planning-level decision chain. Its nodes-only input lacks state and synchronization support, and two fastening actions share a torque tool. The state stage adds 7 support edges, the synchronization stage adds 2 pairings for the cooperative alignment node, and the resource stage adds 1



(a) Compression benefit on the paired linear inputs



(b) List scheduling gap on the same accepted graphs

A: Assembly · I: Inspection · P: Packaging
S: Sorting · T: Workcell preparation · W: Warehouse

Fig. 8. Paired efficiency and same-graph scheduling gaps for 24 tasks. (a) Per-task speedup of the full method and the no-compression ablation on controlled linear-order inputs. (b) Makespan gaps of list scheduling relative to MILP optima on accepted graphs from the original plans. The panels share task identifiers but use different input-relation conditions; the horizontal axis does not represent independent repeated sampling. A, I, P, S, T, and W denote assembly, inspection, packaging, sorting, workstation preparation, and warehousing, respectively; 01–04 identify four tasks in each domain. The 10 zero gaps in (b) are explicitly labeled 0.

order edge. Positioning the base and preparing the bracket run in parallel over [0, 2); alignment B exclusively occupies both units over [2, 5), followed sequentially by fastening, inspection, and labeling. Makespan falls from 14 for fully serial execution of the same nodes to 12. This case explains how relation evidence constrains the release of parallelism; it is not a physical execution record.

Formal gates cannot check unrepresented procedural facts. For example, I02, electrical-module inspection, uses inspection results recorded as the prerequisite for entering the qualified area, without explicitly representing inspection passed or a wire-disconnection step. Although the joint gates pass, this content still requires independent verification of procedural semantics. Together with the field perturbations in Table 5, this example shows that acceptance rate cannot replace correctness in completing a natural-language task.

4.8 Repair comparisons on identical inputs and publicly archived candidates

Direct auditing, one-pass field matching, one-pass field matching with identical compression, and the full method are compared using the same candidate graph, explicit initial state and goals, durations, resources, Cand, and mode. All four share the final audit and strict scheduling and do not read the reference graph. Simple matching selects the first producer according to the canonical node key, removes mislabeled relations, reconstructs synchronization, and orients resources according to the current topological order. The shared-compression combination additionally invokes the full method's compression module. This simple baseline separates the effects of compression and relation selection; it is not a published state-of-the-art method.

Table 8. Relation-repair comparisons on the same 160 schema-constrained candidates.

Setting	Strictly accepted / 160	Full-label edge F1 all 160 records	Accepted-plan speedup
Direct auditing	24	0.9018±0.0807	1.1153±0.1210
One-pass field matching	160	0.9853±0.0256	1.1629±0.1745
One-pass field matching + identical compression	160	0.9853±0.0256	1.2619±0.1242
Full method	160	0.9813±0.0297	1.2535±0.1347

Note: Continuous metrics are per-record means $\pm$ sample standard deviations. Direct-audit speedup covers only the 24 accepted records. The other three settings share 160 accepted inputs and can be compared pairwise. The shared-compression combination and the full method use the same compression, auditing, and scheduling implementations.

All three repair settings strictly accept 160/160 inputs. The simple shared-compression combination achieves edge F1 of 0.9853 and speedup of 1.2619, compared with 0.9813 and 1.2535 for the full method. Makespans are identical in 153 cases and shorter for the simple combination in 7. Edge F1 excludes E_order, and enabling compression does not change the scored edge sets in this batch; thus, the edge-F1 difference cannot be interpreted as the cost of compression. A concrete resource-orientation counterexample is provided in supplementary Section C.6. All inputs in this batch are accepted after repair, and compression improves scheduling, but the full heuristic provides no additional gain.

For the four relation views of the 24 GPT-6 plans, all three repair settings accept 24/24. All-relations, nodes-only, and state-only inputs each yield a mean speedup of 1.2411. For linear-order inputs, one-pass matching yields 1.0000, while both the

shared-compression combination and the full method yield 1.2388. These views share the same 24 base tasks, and the gains mainly arise from deleting conservative ordering; they do not provide additional independent task evidence.

The external evaluation uses TaskBench tool-planning data and the HuggingFace subset and archived direct-inference outputs from the official GNN4TaskPlan repository [39, 56], pinned to commit f1486690f4fa00030bc7035f0dcc41904889aa2c. The data, task references, and model candidates were released before this conversation. This study replays public outputs without invoking the archived models again. Public files and adaptation rules are retained with the reproduction materials.

This external task domain provides no annotations for robot states, resources, or B occupancy. Adaptation reads calls and parameters only from the candidate itself: explicit $\langle \text{node-k} \rangle$ parameter references become producer-consumer states, literal parameters become explicit initial facts, and candidate task_links become state relations to be checked. The tool dictionary is used only for name validation and additional type diagnostics. Each call has duration 1, empty Res, mode=single, and Cand={L,R}; the goals are the outputs of all candidate calls. Acceptance therefore means only that the parameter dependencies of this set of calls are closed. It does not establish completion of the user's natural-language goal or support claims about physical parallel efficiency. Duplicate tool names, unknown tools, invalid containers, and references that cannot be uniquely resolved are rejected before adaptation, without guessed completion.

Table 9. Coverage, formal acceptance, and external-reference agreement of publicly archived candidates.

Archived model	Available archives /test cases	Adapt- able	Strictly accepted before $\rightarrow$ after	External edge F1 before $\rightarrow$ after	Accepted and exact reference match
CodeLlama-13b	497/500	234	175 $\rightarrow$ 234	0.4226 $\rightarrow$ 0.4040	53 $\rightarrow$ 53
mistral-7b	489/500	67	44 $\rightarrow$ 64	0.4064 $\rightarrow$ 0.3023	8 $\rightarrow$ 10

Note: Each model's test set contains 500 cases, with 3 and 11 missing archived outputs, respectively. Adaptation fails for 263 and 422 cases; all remain in the available-archive denominators and are not counted as accepted. Edge F1 compares directed node pairs identified by tool names and is averaged over all 234 or 67 adaptable records, including those ultimately rejected. It is not the full-label edge F1 used in the main experiment. Exact reference match requires equality of both the tool-node set and the directed-edge set and does not cover parameter values or actual execution.

For CodeLlama, formal acceptance rises from 175 to 234 while external edge F1 falls from 0.4226 to 0.4040. For Mistral, acceptance rises from 44 to 64 and edge F1 falls from 0.4064 to 0.3023. The full method improves 15 and 4 cases, respectively, and worsens 13 and 12. One-pass field matching and its shared-compression combination

both accept 234 and 64 cases and yield post-repair edge F1 of 0.4040 and 0.3033, respectively, showing no advantage of the full method in external relation quality. Only 53 and 10 records are both accepted and exact matches to the reference nodes and edges; these counts are not task-success rates.

In the public tool archives, parameter references can be internally consistent yet depart from the task reference. Increased formal acceptance therefore cannot substitute for improved reference correctness. Section 4.9 further examines material, resource, and cooperative obligations using public source specifications. The constraints covered by the two kinds of public evidence are interpreted separately.

4.9 Discrete projections of public specifications and mechanism boundaries

To address the lack of resource and cooperative annotations in tool tasks, the evaluation uses public assembly recipes and specification tests from AssemblyGrid v1 [4] and job-shop instances from OR-Library [5]. The former is pinned to commit ce61f635e47d312d633eda022b5f08bc4dbe888f; the latter is pinned to jobshop1.txt and its hash. The adaptation protocol, selection scope, and inputs were frozen before evaluation. No model is used to generate references, and no relations are randomly deleted.

Of the 10 AssemblyGrid recipes, 6 can be fully projected onto two execution units, comprising 15 nodes in total; 3 recipes contain 4 cooperative nodes. The other 4 are excluded because they require more than 2 cooperating units or contain unresolved exclusive routes. Tasks are not reduced, and required team sizes are not lowered. One three-node inclusive-OR case from the official tests is also included; both routes produce the same required material. Its exclusive-XOR counterpart is not adapted. This case is a public specification test, not an additional official recipe.

Recipe adaptation preserves raw materials, material inputs and outputs, durations, explicit precedence, and required numbers of cooperating units. Candidate relations are converted only from explicit precedence fields, while material obligations remain in node fields. Repair therefore addresses obligations not yet expressed by the relation conversion. The two base execution units are assumed to possess the required skills, and raw materials are ready. Geometry, transport, spatial occupancy, and skill matching in the original simulation are not evaluated. Capacity-resource evaluation uses the preselected ft06 and la01–la05 instances, preserving their machines, processing times, and within-job ordering and adding two general-purpose base execution units. These are dual-execution-unit-constrained projections of the original job-shop problems, not rankings against the original benchmark optima.

The four settings retain the definitions and identical information access of Section 4.8. Settings requiring compression share the same module, final audit, and strict scheduling. An independent verifier reads the public source recipes or job matrices directly rather than treating repaired Pre, Post, or Res as ground truth. It checks operation coverage, original durations, material production and consumption, source precedence obligations, capacity resources, cooperative team size, L/R occupancy, and source goals. The 52 evaluations comprise $13 \text{ inputs} \times 4 \text{ settings}$ and produce 41 accepted outputs, all of which pass source-specification verification.

Table 10. Strict acceptance and source-constraint verification for public-source specification projections.

Input group	Inputs	Direct auditing	Simple matching + compression	Full method	Full-method source verification
Public recipes	6	2/6	6/6	6/6	6/6
Public OR test	1	0/1	1/1	1/1	1/1
Job-shop projections	6	0/6	6/6	6/6	6/6

Note: One-pass matching without compression also accepts every input in all three groups. Denominators are the numbers of adaptable inputs. Recipe coverage is 6/10, and excluded cases are included in the package. Source verification covers only constraints retained by the projection; it does not establish success in the original simulation or natural-language semantic accuracy.

Table 11. Resource-orientation comparison with identical inputs and shared compression.

Instance	Nodes	Re-sources	Simple-combination makespan	Full-method makespan	Reduction
ft06	36	6	152	113	25.66%
la01	50	5	2272	1706	24.91%
la02	50	5	2025	1620	20.00%
la03	50	5	1732	1368	21.02%
la04	50	5	2195	1423	35.17%
la05	50	5	1779	1289	27.54%

Note: Makespan uses the symbolic duration units of the source data. The simple combination is one-pass field matching + identical compression. Reduction is (simple-combination makespan – full-method makespan)/simple-combination makespan.

For the recipes and OR test, the three repair settings yield identical makespans in each case. The full method is faster on all 6 resource instances, with a mean per-instance reduction of 25.72%. Mean speedups relative to fully serial execution of the same nodes are 1.7200 and 1.2761 for the full method and the simple combination, respectively. Paired outputs have identical non-resource relations and contain no E_order edges. The difference therefore arises from resource orientation, not order compression or producer selection. The gain is confined to the tested resource instances. The controlled inputs in Table 8 show no such advantage, so these results do not establish a general advantage or statistical significance.

A serial-permutation enumerator independent of the repair algorithm further evaluates a three-node monotonic-state family. Each node's precondition is selected from

ready, x, and y, and its postcondition from x and y. Two duration sets, the presence or absence of B, and shared or separate resources are enumerated, giving 1728 mechanism-test records from 216 logical templates. The initial state is {ready}, and the goals comprise all postcondition states. Exhaustive enumeration of the six node orders identifies feasible witnesses for 736 records. Simple matching and its compression combination accept 672 records. Adding acyclic producer filtering to the simple combination yields 720 accepted records, as does the full method. No setting falsely accepts a record, while each of the latter two still rejects 16 inputs with feasible witnesses. This author-constructed diagnostic shows the role of local acyclic filtering and the incompleteness of repair. It is not independent industrial-task evidence and shows no additional benefit of critical-path producer ranking over an acyclic simple rule.

4.10 Diverse task planning and dual-arm physics simulation

Accepted plans drive two Panda arms in MuJoCo 3.3.3 physics simulation using franka_description 0.7.1 models [[CITE_NEEDED]]. The setup uses a 2 ms integration step, a 20 ms control period, gravity compensation, position actuation, inverse kinematics, and fixed Cartesian waypoints. Objects move through contact and friction, without welding, attachment, or pose resets after initialization. This section evaluates execution of the planning-level interface with given skills; it does not propose a new motion-planning or grasping algorithm.

Eight task templates are constructed, as listed in Table 12. T01 permits independent pick-and-place actions. T03 additionally requires both objects to be grasped before placement, creating a state join, although equal durations can produce identical windows. T02 and T04 include one-stage and two-stage service at a shared capacity-1 workstation, respectively. Workstation service is represented by entering a designated region and dwelling for at least 0.5 s; inspection sensors are not simulated. T05 and T06 contain one B transport and two consecutive B transports, respectively. T07 and T08 connect tray loading, transport, and unloading. Node fields, skills, seeds, and criteria were frozen before the formal evaluation. Task specifications and relation defects are author-constructed and are not independent natural-language plans or natural LLM errors.

Nominal conditions use seeds 100–104 and ± 10 mm planar perturbations to initial object positions. The two schedules share the accepted graph, assignment, and fixed skills; the serial control only removes concurrent execution. Both complete 40/40 runs. The proposed schedule shortens windows by 14.71%–50.00% across templates (Fig. 10a), representing gains from overlap at the given skill durations. Skill durations and video-selection rules are specified in supplementary Section A.6. Fig. 9 uses actual recordings from the preselected seed 100.

The same-input relation-repair comparison applies four relation conditions to the eight templates, producing 32 inputs and 96 method settings (Table 13 and Fig. 10b). Fields-only inputs contain no relations. Mislabeled inputs replace state-support and synchronization labels with mismatching states. Conservative-order inputs contain valid state support and a topologically valid serial chain. Missing-producer inputs remove one node that produces a goal. An independent monotonic-state reachability check confirms that goals are reachable for 24 inputs and unreachable for the other 8.

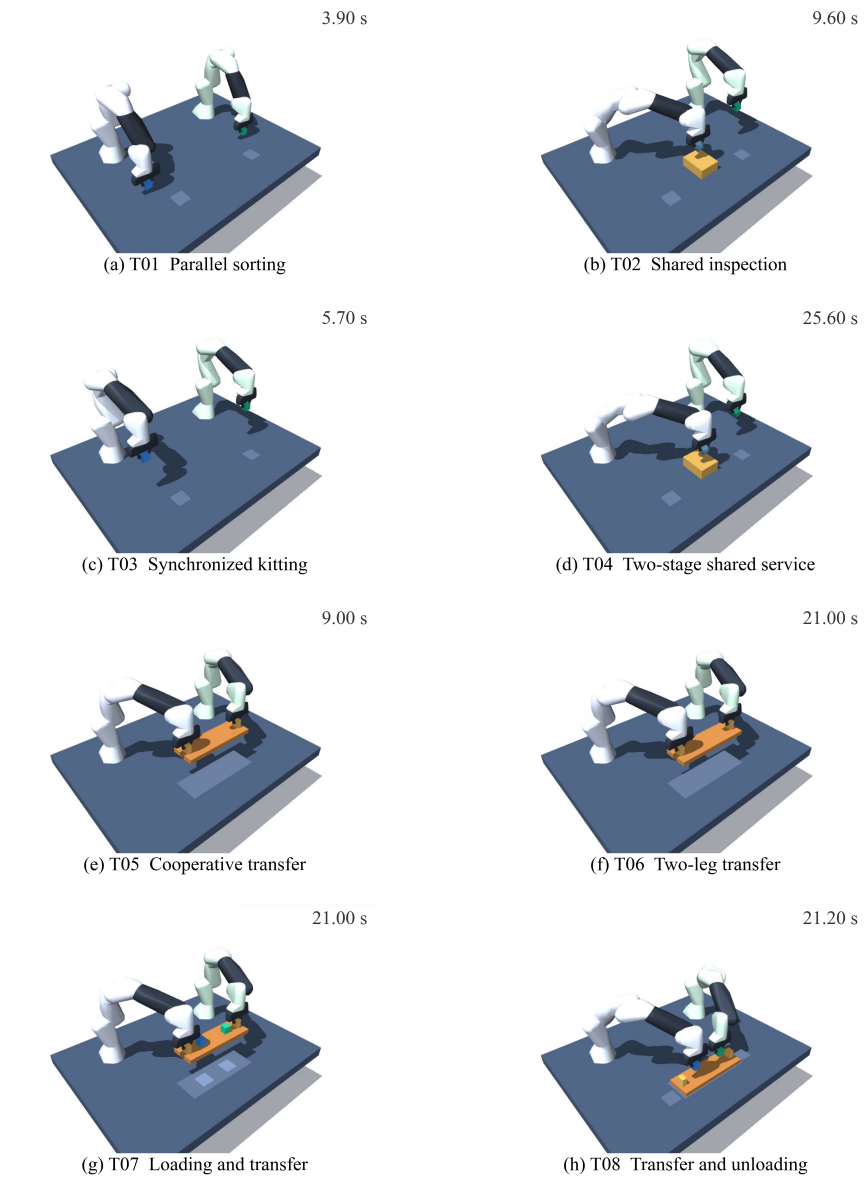


Fig. 9. Actual physics-simulation states for eight task templates. Panels correspond to T01–T08 in Table 12, and times follow the simulation clock. Frames are from proposed-method recordings with seed 100, showing grasping, workstation service, cooperative transport, and unloading; they are not photographs of physical robots.

Table 12. Eight task classes, nominal schedule windows, and object-position errors.

Code and task	Nodes	B nodes	Serial win- dow/s	Proposed window/s	Maximum error/mm
T01 Parallel sorting	4	0	24	12	7.34
T02 Shared inspection workstation	6	0	40	28	16.59
T03 Synchronized kit- ting	4	0	24	12	7.34
T04 Two-stage shared service	8	0	56	44	26.10
T05 Cooperative trans- port	5	1	22	17	1.05
T06 Two-stage transport	6	2	34	29	1.08
T07 Tray loading fol- lowed by transport	9	1	46	29	7.02
T08 Transport followed by unloading	9	1	46	29	7.11

Note: Each template has five initial-state configurations, with 5 proposed-schedule and 5 serial runs, all successful (5/5), for 80 runs in total. Windows are simulated schedule spans using the given skill durations. Maximum error is the largest final Euclidean error across all target objects in the five proposed-method runs. The five initial states are not five independent task templates.

Audit only, one-pass field matching with identical compression, and the full method have identical field access. Every accepted output undergoes independent relation and schedule checks.

Both repair methods accept 24/32 inputs, with identical schedule windows for all 24 pairs; audit only accepts 8/32. Repair restores relations derivable from the fields, but the full heuristic does not improve on simple rules with shared compression on these inputs. All three settings reject inputs lacking a goal producer because adding edges cannot supply a missing action.

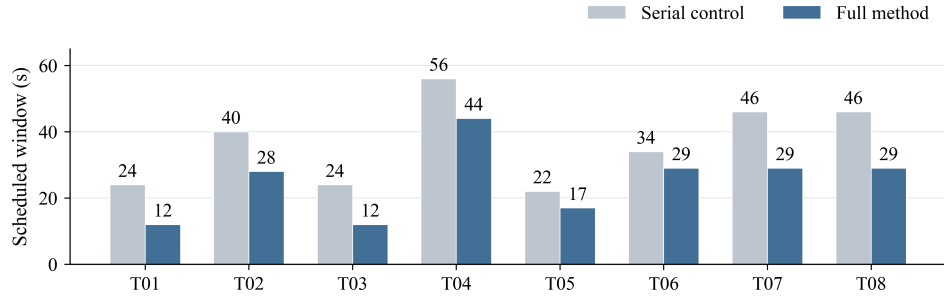
Physical stress tests apply one-factor changes using the same five seeds and execute only the proposed schedule, for 160 runs in total: object mass multiplied by 5; sliding friction reduced from 1.5 to 0.15; ± 20 mm planar bias added to the controller's initial grasp-position estimate; or all skill windows halved. All other model and control parameters remain frozen, with no tuning based on formal evaluation results. Position bias is not an evaluation of a visual-perception system, and given target positions other than grasp targets are still supplied by the task specification. Table 14 reports planning acceptance and physical completion separately; failed runs remain in the denominators.

Success requires all action postconditions and discrete goals to hold, every object's final position error to be below 30 mm, and its linear speed to be below 0.02 m/s. Shared-workstation occupancy, inter-arm penetrating contacts deeper than 1 mm, and

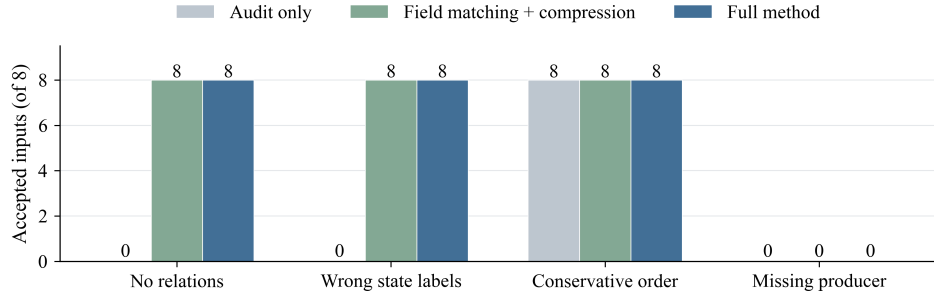
Table 13. Strict acceptance counts on the same constructed inputs (8 per condition).

Input condition	Audit only	Matching + identical compression	Full method
Node fields only	0/8	8/8	8/8
Mislabeled semantic relations	0/8	8/8	8/8
Conservative-order chain	8/8	8/8	8/8
Missing goal producer	0/8	0/8	0/8

Note: The 32 inputs are derived from eight templates, not 32 independent semantic tasks. All 56 accepted settings pass independent checks. Unreachability in the missing-producer group applies only to its represented monotonic-state model.



(a) Fixed-skill schedule windows under nominal conditions



(b) Same-input repair controls on constructed candidates

Fig. 10. Fixed-skill schedule windows and same-input repair comparisons. (a) Nominal serial and proposed windows for eight templates; paired executions share the graph, assignment, and skills. (b) Strict acceptance counts for four constructed input conditions, each with denominator 8. Zero values are explicitly labeled. The figure reports neither significance nor random-error ranges.

engine warnings are also checked. Grasping explicitly checks lift height, gripper distance, and contact with both fingers. Approach and retreat check end-effector arrival and gripper opening; retreat additionally checks clearance from the object. During the designated carrying interval of a B transport, each arm must have both-finger contact in more than 90% of physics steps. Any action failure stops subsequent dispatch, while trajectories and failure witnesses are retained.

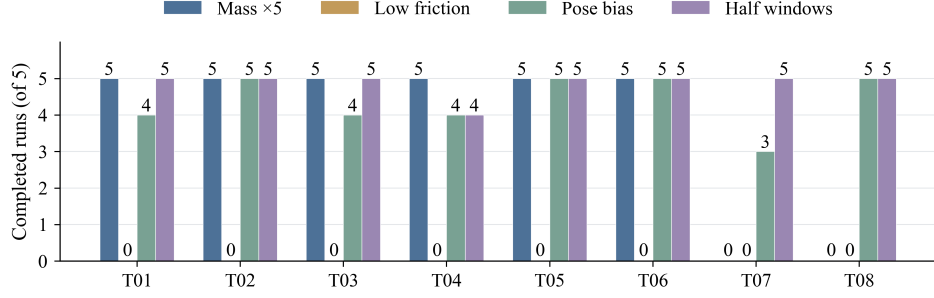
Table 14. Planning acceptance and physical completion under all single-factor conditions (proposed schedule).

Condition	Parameter	Planning acceptance	Physical completion
Nominal	Baseline parameters	40/40	40/40
Increased mass	Mass $\times$ 5	40/40	30/40
Low friction	Sliding coefficient 0.15	40/40	0/40
Position bias	Initial estimate $\pm$ 20 mm	40/40	35/40
Half duration	All skill windows $\times$ 0.5	40/40	39/40

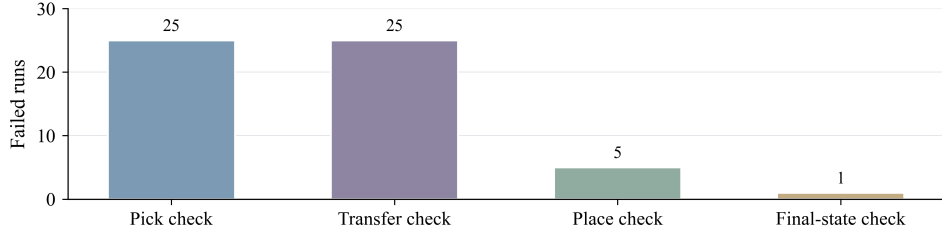
Note: Each row has a denominator of eight templates $\times$ five seeds. The nominal row is the same 40 proposed-schedule runs as in Table 12. An additional 40 nominal serial runs bring the total to 240 physical runs. Planning acceptance does not include payload, friction, or grasp geometry and cannot be treated as a prediction of physical success.

Of 240 runs, 184 complete and 56 fail. An independent program reconstructs every action endpoint from saved generalized coordinates, velocities, and control inputs and rechecks dwell and carrying contact using trajectories sampled at 40 ms. Its decisions agree with the online results in 240/240 cases, without directly using the event success flags. Increased mass causes 10 loaded-transport failures in T07 and T08; low friction causes all 40 runs to fail; position bias and half duration cause 5 and 1 failures, respectively. Fig. 11 classifies each run by its first failed check: 25 grasping, 25 cooperative-transport, 5 placement, and 1 final-state failures. A run may also trigger other checks; this classification does not claim to exhaust the root causes.

Accepted plans complete fixed-skill execution under the tested nominal parameters but can still fail under stress. Without a serial control under stress, the failure rates cannot be attributed to parallel scheduling. The author-constructed templates, repeated seeds, and shared controller do not establish robustness on other tasks or transfer to physical robots. Online counts at 2 ms and offline samples at 40 ms report the checks at their respective resolutions and cannot prove continuous-time collision freedom.



(a) One-factor stress tests using full scheduling



(b) First failed check in each of the 56 failed runs

Fig. 11. Completion under fixed-skill perturbations and the distribution of failures. (a) Five runs per task and condition; all zero-completion cases under low friction are explicitly labeled. (b) The 56 failures are counted in mutually exclusive categories according to the first failed check. Subsequent unexecuted nodes are not counted as additional failure samples.

5 Discussion and conclusions

The method applies state closure, synchronization pairing, resource orientation, and conservative order compression to a candidate atomic task graph. Joint graph auditing and actual schedule checks then determine whether to return an accepted plan or diagnostics. Reasons are recorded for both accepted and rejected edits, and accepted events can be replayed to reconstruct the refined graph.

In the controlled experiments, repairing state, synchronization, and resource relations improves acceptance, while conservative order compression allows more tasks to run in parallel. Simple rules with shared compression nevertheless achieve slightly higher edge F1 and speedup in the main experiment and the same windows as the full method on the 24 acceptable simulation-template inputs. The full method's relation-selection heuristic provides no additional gain on these two sets of inputs.

Across six dual-unit projections of public job-shop specifications, the full method reduces makespan by a mean of 25.72% per instance. Paired outputs have identical non-resource relations and no conservative order edges, attributing the difference to resource orientation. The strategy helps under these tested resource-competition conditions, but this does not establish a general advantage. MILP still obtains shorter

times on fixed accepted graphs, so passing the relation checks does not imply an optimal schedule.

All original GPT-6 candidates from the method-informed conversation pass with no net relation changes, so this batch cannot estimate gains from repairing naturally generated errors. In public tool archives, acceptance increases while reference-edge F1 declines, showing that incorrect fields can still form internally consistent plans. Public-specification projections add independent constraints, but evaluation remains limited to the material, resource, and cooperative requirements retained by the projection.

Across eight simulation templates, all 40 nominal parallel runs and all 40 nominal serial runs complete, compared with 104/160 runs under stress. Friction, payload, position estimates, and skill durations are outside the discrete acceptance criteria, allowing accepted plans to fail during execution. The stress tests have no serial control, so they cannot determine whether the failure rate is related to parallel scheduling.

The guarantee is limited to the checks on represented monotonic discrete states, capacity-1 resources, two base execution units, and numerical intervals. Small-scale exhaustive enumeration still finds feasible inputs that are rejected, showing that repair is incomplete. Fixed-skill simulation also cannot establish continuous collision freedom or transfer to physical robots. Further work requires independent procedural specifications and independently generated candidates to test field semantics, together with a way to use physical execution feedback in discrete planning checks.

Statements and Declarations

Data and code availability

The frozen planning package `ATG_reproduction_v7_70` and extended simulation package `simulation_expanded` retain raw inputs, per-record results, failure logs, independent verifiers, versions, and SHA-256 manifests. Third-party materials retain their source and license information. Detailed protocols, reproduction commands, and the video index are provided in supplementary Appendix A. Code and data are available at <https://github.com/RoboPlanner/atg-repair-planning>; the project page is <https://roboplanner.github.io/atg-repair-planning/>. Historical experiments retain their archived environments, and recomputed results are not relabeled as new timings or model calls.

Declaration of generative AI and AI-assisted technologies

OpenAI Codex was used to assist manuscript organization, editing, Chinese-to-English translation, and the preparation and revision of implementation, analysis, and plotting code. The present translation was produced in a session identified as GPT-6. OpenAI ImageGen was used for initial visual drafts for Figs. 1–4 and retained object illustrations in Figs. 1, 3, and 4; its backend model version was not recorded. The final explanatory diagrams were reconstructed and edited in PowerPoint. GPT-6 candidate generation is a research input, separately documented in Section 4.5 and Supplementary Appendix A; it is not an independently verifiable API benchmark.

References

- [1] Agia C, Jatavallabhula KM, Khodeir M, et al (2022) Taskography: Evaluating robot task planning over large 3d scene graphs. In: Conference on Robot Learning. PMLR, pp 46–58, <https://doi.org/10.48550/arXiv.2207.05006>
- [2] Alayrac JB, Donahue J, Luc P, et al (2022) Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35:23716–23736. <https://doi.org/10.52202/068431-1723>
- [3] Anil R, Dai AM, Firat O, et al (2023) PaLM 2 technical report. *arXiv preprint arXiv:230510403* <https://doi.org/10.48550/arXiv.2305.10403>
- [4] Bahrpeyma F, Heik D, Reichelt D (2026) AssemblyGrid v1: A Benchmark for Multi-Robot Production with Temporary Coalitions, Local Information, and Geometric Constraints. *arXiv preprint arXiv:260916075* <https://doi.org/10.48550/arXiv.2609.16075>, URL <https://arxiv.org/abs/2609.16075>
- [5] Beasley JE (1990) OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society* 41(11):1069–1072. URL <https://people.brunel.ac.uk/~mastjjb/jeb/info.html>
- [6] Besta M, Blach N, Kubicek A, et al (2024) Graph of thoughts: Solving elaborate problems with large language models. In: *Proceedings of the AAAI conference on artificial intelligence*, pp 17682–17690, <https://doi.org/10.1609/aaai.v38i16.29720>
- [7] Bhat V, Kaypak AU, Krishnamurthy P, et al (2024) Grounding llms for robot task planning using closed-loop state feedback. *arXiv preprint arXiv:240208546* <https://doi.org/10.48550/arXiv.2402.08546>
- [8] Cao Y, Han S, Gao Z, et al (2025) GraphInsight: Unlocking Insights in Large Language Models for Graph Structure Understanding. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, pp 12096–12134, <https://doi.org/10.18653/v1/2025.acl-long.591>, URL <https://aclanthology.org/2025.acl-long.591/>
- [9] Chen L, Lu K, Rajeswaran A, et al (2021) Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems* 34:15084–15097. <https://doi.org/10.48550/arxiv.2106.01345>
- [10] Chen Y, Wu T, Wang S, et al (2022) Towards human-level bimanual dexterous manipulation with reinforcement learning. *Advances in Neural Information Processing Systems* 35:5150–5163. <https://doi.org/10.52202/068431-0372>
- [11] Chi C, Xu Z, Feng S, et al (2025) Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research* 44(10-11):1684–1704. <https://doi.org/10.1177/02783649241273668>, URL <https://journals.sagepub.com/doi/10.1177/02783649241273668>
- [12] Cui Y, Xu Z, Zhong L, et al (2025) A Task-Adaptive Deep Reinforcement Learning Framework for Dual-Arm Robot Manipulation. *IEEE Transactions on Automation Science and Engineering* 22:466–479. <https://doi.org/10.1109/TA>

- SE.2024.3352584, URL <https://doi.org/10.1109/TASE.2024.3352584>
- [13] Driess D, Xia F, Sajjadi MSM, et al (2023) PaLM-E: an embodied multimodal language model. In: Proceedings of the 40th International Conference on Machine Learning, pp 8469–8488, <https://doi.org/10.48550/arXiv.2303.03378>
 - [14] Fan L, Wang G, Jiang Y, et al (2022) Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems* 35:18343–18362. <https://doi.org/10.48550/arxiv.2206.08853>
 - [15] Goeckner A, Sui Y, Martinet N, et al (2024) Graph neural network-based multi-agent reinforcement learning for resilient distributed coordination of multi-robot systems. In: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, pp 5732–5739, <https://doi.org/10.48550/arXiv.2403.13093>
 - [16] Grotz M, Shridhar M, Chao YW, et al (2024) PerAct²: Benchmarking and Learning for Robotic Bimanual Manipulation Tasks. In: CoRL 2024 Workshop on Whole-body Control and Bimanual Manipulation: Applications in Humanoids and Beyond, URL <https://openreview.net/forum?id=nIU0ZFmptX>
 - [17] Gu Q, Kuwajerwala A, Morin S, et al (2024) Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). IEEE, pp 5021–5028, <https://doi.org/10.1109/ICRA57147.2024.10610243>
 - [18] Hafidi H, Ghogho M, Ciblat P, et al (2020) GraphCL: Contrastive self-supervised learning of graph representations. *arXiv preprint arXiv:200708025* <https://doi.org/10.48550/arXiv.2007.08025>
 - [19] Hao S, Gu Y, Ma H, et al (2023) Reasoning with Language Model is Planning with World Model. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp 8154–8173, <https://doi.org/10.18653/v1/2023.emnlp-main.507>
 - [20] Howey R, Long D, Fox M (2004) VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In: 16th IEEE International Conference on Tools with Artificial Intelligence. IEEE Computer Society, pp 294–301, <https://doi.org/10.1109/ICTAI.2004.120>, URL <https://doi.org/10.1109/ICTAI.2004.120>
 - [21] Hu Y, Lin F, Zhang T, et al (2023) Look before you leap: Unveiling the power of GPT-4V in robotic vision-language planning. *arXiv preprint arXiv:2311.17842* <https://doi.org/10.48550/arXiv.2311.17842>
 - [22] Huang W, Xia F, Xiao T, et al (2023) Inner Monologue: Embodied Reasoning through Planning with Language Models. In: Conference on Robot Learning. PMLR, pp 1769–1782, <https://doi.org/10.48550/arXiv.2207.05608>
 - [23] Huang X, Liu W, Chen X, et al (2024) Understanding the planning of LLM agents: A survey. *arXiv preprint arXiv:240202716* <https://doi.org/10.48550/arXiv.2402.02716>

- [24] Ichter B, Brohan A, Chebotar Y, et al (2023) Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In: Proceedings of The 6th Conference on Robot Learning, vol 205. PMLR, pp 287–318, URL <https://proceedings.mlr.press/v205/ichter23a.html>
- [25] Jang E, Irpan A, Khansari M, et al (2022) BC-Z: Zero-Shot Task Generalization with Robotic Imitation Learning. In: Proceedings of the 5th Conference on Robot Learning, vol 164. PMLR, pp 991–1002, URL <https://proceedings.mlr.press/v164/jang22a.html>
- [26] Kipf TN, Welling M (2017) Semi-Supervised Classification with Graph Convolutional Networks. In: International Conference on Learning Representations, URL <https://openreview.net/forum?id=SJU4ayYgl>
- [27] Liang J, Huang W, Xia F, et al (2023) Code as Policies: Language Model Programs for Embodied Control. In: IEEE International Conference on Robotics and Automation. IEEE, pp 9493–9500, <https://doi.org/10.1109/icra48891.2023.10160591>
- [28] Lin F, La Malfa E, Hofmann V, et al (2024) Graph-enhanced Large Language Models in Asynchronous Plan Reasoning. In: Proceedings of the 41st International Conference on Machine Learning, vol 235. PMLR, pp 30108–30134
- [29] Liu H, Li C, Li Y, et al (2024) Improved baselines with visual instruction tuning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 26296–26306
- [30] Luo L, Li Y, Haffari G, et al (2024) Reasoning on Graphs: Faithful and Interpretable Large language Model Reasoning. In: ICLR 2024: The Twelfth International Conference on Learning Representations. ICLR
- [31] McDermott D, Ghallab M, Howe A, et al (1998) PDDL - The Planning Domain Definition Language. Technical report cvc tr-98-003/dcs tr-1165, Yale Center for Computational Vision and Control, New Haven, CT, URL <https://www.cs.cmu.edu/~mmv/planning/readings/98aips-PDDL.pdf>
- [32] Ning X, Zhao Y, Liu Y, et al (2024) DGoT: Dynamic Graph of Thoughts for Scientific Abstract Generation. In: Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), pp 4832–4846, <https://doi.org/10.63317/2omc23hakttj>
- [33] Nourmohammadi A, Fathi M, Ng AHC (2024) Balancing and scheduling human-robot collaborated assembly lines with layout and objective consideration. Computers & Industrial Engineering 187:109775. <https://doi.org/10.1016/j.cie.2023.109775>, URL <https://doi.org/10.1016/j.cie.2023.109775>
- [34] OpenAI, Achiam J, Adler S, et al (2023) GPT-4 technical report. arXiv preprint [arXiv:230308774](https://arxiv.org/abs/230308774)
- [35] Pramanick P, Barua HB, Sarkar C (2020) Decomplex: Task planning from complex natural instructions by a collocating robot. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, pp 6894–6901, <https://doi.org/10.1109/iros45743.2020.9341289>

- [36] Qi Z, Ma M, Xu J, et al (2025) Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solver. In: The Thirteenth International Conference on Learning Representations
- [37] Qiao S, Fang R, Zhang N, et al (2024) Agent planning with world knowledge model. *Advances in Neural Information Processing Systems* 37:114843–114871. <https://doi.org/10.52202/079017-3646>
- [38] Raspanti F, Ozcelebi T, Holenderski M (2025) Grammar-constrained decoding makes large language models better logical parsers. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp 485–499, <https://doi.org/10.18653/v1/2025.acl-industry.34>
- [39] Shen Y, Song K, Tan X, et al (2024) TaskBench: Benchmarking Large Language Models for Task Automation. In: *Advances in Neural Information Processing Systems*, vol 37. Curran Associates, Inc., pp 4540–4574, <https://doi.org/10.52202/079017-0148>, URL https://proceedings.neurips.cc/paper_files/paper/2024/file/085185ea97db31ae6dcac7497616fd3e-Paper-Datasets_and_Benchmarks_TaskBench.pdf
- [40] Shinn N, Cassano F, Gopinath A, et al (2023) Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems* 36:8634–8652. <https://doi.org/10.52202/075280-0377>, URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [41] Shridhar M, Manuelli L, Fox D (2023) Perceiver-actor: A multi-task transformer for robotic manipulation. In: *Conference on Robot Learning*. PMLR, pp 785–799, <https://doi.org/10.48550/arXiv.2209.05451>
- [42] Singh I, Blukis V, Mousavian A, et al (2023) ProgPrompt: Generating situated robot task plans using large language models. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp 11523–11530, <https://doi.org/10.1109/icra48891.2023.10161317>
- [43] Stechly K, Valmeekam K, Kambhampati S (2025) ON THE SELF-VERIFICATION LIMITATIONS OF LARGE LANGUAGE MODELS ON REASONING AND PLANNING TASKS. In: *13th International Conference on Learning Representations, ICLR 2025*. International Conference on Learning Representations, ICLR, pp 8097–8150, <https://doi.org/10.48550/arXiv.2402.08115>
- [44] Sui Y, Zhou M, Zhou M, et al (2024) Table meets llm: Can large language models understand structured table data? a benchmark and empirical study. In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pp 645–654, <https://doi.org/10.1145/3616855.3635752>
- [45] Sun H, Zhuang Y, Kong L, et al (2023) AdaPlanner: Adaptive Planning from Feedback with Language Models. *Advances in Neural Information Processing Systems* 36:58202–58245. <https://doi.org/10.52202/075280-2537>, URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/b5c8c1c117618267944

- [46] Sun J, Xu C, Tang L, et al (2024) Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In: The Twelfth International Conference on Learning Representations, <https://doi.org/10.48550/arXiv.2307.07697>
- [47] Sun W, Li J, Chen L, et al (2024) Rethinking and simplifying bootstrapped graph latents. In: Proceedings of the 17th ACM International Conference on Web Search and Data Mining, pp 665–673, <https://doi.org/10.1145/3616855.3635842>
- [48] Surís D, Menon S, Vondrick C (2023) ViperGPT: Visual inference via python execution for reasoning. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 11888–11898
- [49] Touvron H, Lavril T, Izacard G, et al (2023) LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:230213971 <https://doi.org/10.48550/arXiv.2302.13971>
- [50] Valmeekam K, Marquez M, Sreedharan S, et al (2023) On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems* 36:75993–76005
- [51] Veličković P, Cucurull G, Casanova A, et al (2018) Graph Attention Networks. In: International Conference on Learning Representations, URL <https://openreview.net/forum?id=rJXMpikCZ>
- [52] Venkatesh SG, Upadrashta R, Amrutur B (2021) Translating natural language instructions to computer programs for robot manipulation. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, pp 1919–1926, <https://doi.org/10.48550/arxiv.2012.13695>
- [53] Virtanen P, Gommers R, Oliphant TE, et al (2020) SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17(3):261–272. <https://doi.org/10.1038/s41592-019-0686-2>, URL <https://doi.org/10.1038/s41592-019-0686-2>
- [54] Wang L, Xu W, Lan Y, et al (2023) Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 2609–2634, <https://doi.org/10.18653/v1/2023.acl-long.147>, URL <https://aclanthology.org/2023.acl-long.147/>
- [55] Wen Y, Wang Z, Sun J (2024) MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. In: 62nd Annual Meeting of the Association for Computational Linguistics, ACL 2024. Association for Computational Linguistics (ACL), pp 10370–10388, <https://doi.org/10.18653/v1/2024.acl-long.558>
- [56] Wu X, Shen Y, Shan C, et al (2024) Can Graph Learning Improve Planning in LLM-based Agents? In: *Advances in Neural Information Processing Systems*, vol 37. Curran Associates, Inc., pp 5338–5383, <https://doi.org/10.52202/079017-0173>, URL https://proceedings.neurips.cc/paper_files/paper/2024/file/098d

- 1bd3eb6156a4c2f834563cdcf617-Paper-Conference.pdf
- [57] Wu Z, Han C, Ding Z, et al (2024) OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. In: ICLR 2024 Workshop on Large Language Model (LLM) Agents, URL <https://openreview.net/forum?id=3WWFrg8UjJ>
 - [58] Yao S, Chen H, Yang J, et al (2022) Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems* 35:20744–20757. <https://doi.org/10.52202/068431-1508>
 - [59] Yao S, Yu D, Zhao J, et al (2023) Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36:11809–11822
 - [60] Yao Y, Li Z, Zhao H (2024) GoT: Effective graph-of-thought reasoning in language models. In: *Findings of the Association for Computational Linguistics: NAACL 2024*, pp 2901–2921, <https://doi.org/10.18653/v1/2024.findings-naacl.183>
 - [61] You Y, Cai B, Pham DT, et al (2025) A human digital twin approach for fatigue-aware task planning in human-robot collaborative assembly. *Computers & Industrial Engineering* 200:110774. <https://doi.org/10.1016/j.cie.2024.110774>, URL <https://doi.org/10.1016/j.cie.2024.110774>
 - [62] Zhang S, Ma X, Cao Z, et al (2025) Plan-over-graph: Towards parallelable llm agent schedule. *arXiv preprint arXiv:250214563* <https://doi.org/10.48550/arXiv.2502.14563>
 - [63] Zhang X, Zitnik M (2020) GNNGuard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems* 33:9263–9275. <https://doi.org/10.48550/arXiv.2006.08149>
 - [64] Zhang Y (2025) A Coordinated Planning and Control Framework for Mobile Dual-arm Robots with Manipulability Optimization and Force Tracking. *Journal of Intelligent & Robotic Systems* 111(2):51. <https://doi.org/10.1007/s10846-025-02247-y>, URL <https://link.springer.com/article/10.1007/s10846-025-02247-y>
 - [65] Zhang Z, Chen J, Yang D (2024) Darg: Dynamic evaluation of large language models via adaptive reasoning graph. *Advances in Neural Information Processing Systems* 37:135904–135942. <https://doi.org/10.52202/079017-4317>
 - [66] Zhao Z, Lee WS, Hsu D (2023) Large language models as commonsense knowledge for large-scale task planning. *Advances in Neural Information Processing Systems* 36:31967–31987. <https://doi.org/10.52202/075280-1387>
 - [67] Zheng C, Li Z, Janardhanan M, et al (2025) Balancing and scheduling human-robot collaborative assembly lines with heterogeneous robots and limited resources: Constraint programming approach and fruit fly optimization algorithm. *Computers & Industrial Engineering* 203:111046. <https://doi.org/10.1016/j.cie.2025.111046>, URL <https://doi.org/10.1016/j.cie.2025.111046>
 - [68] Zhou A, Yan K, Shlapentokh-Rothman M, et al (2024) Language agent tree search unifies reasoning, acting, and planning in language models. In: *Proceedings of*

the 41st International Conference on Machine Learning, pp 62138–62160

- [69] Zitkovich B, Yu T, Xu S, et al (2023) RT-2: Vision-language-action models transfer web knowledge to robotic control. In: Conference on Robot Learning. PMLR, pp 2165–2183, <https://doi.org/10.48550/arxiv.2307.15818>